From Gates to a RISC-V CPU

Hands-on digital design on an FPGA,
with an open-source toolchain

Eighteen lessons, 00–17, all built, tested and measured.
Every number in this book came out of the tools.

Lattice ECP5 `LFE5U-85F` · Radiona ULX3S
Yosys · nextpnr · Project Trellis · Icarus Verilog · Verilator

Setup, commands and the board's pinout: *Tools and Workflow*.

Version 1.0 · 13 November 2026

This book is licensed under Creative Commons Attribution 4.0 International.
The accompanying code is licensed under the MIT License.
Copyright © 2026 The Open FPGA Course.

Contents

The big picture 2
The fabric, and what it costs 4
Lesson 00 — Blinky 8
Lesson 01 — Logic gates 10
Lesson 02 — Combinational logic 12
Lesson 03 — Sequential logic 16
Lesson 04 — Clocks and timing 19
Lesson 05 — Finite state machines 22
Lesson 06 — SPI and the OLED 26
Lesson 07 — Font ROM and coordinate maths 28
Lesson 08 — Memories 30
Lesson 09 — UART 33
Lesson 10 — The ALU 38
Lesson 11 — The register file 41
Lesson 12 — Decode and control 44
Lesson 13 — A single-cycle RV32I processor 47
Lesson 14 — Formal verification 55
Lesson 15 — A system on chip 58
Lesson 16 — Pipelining 63
Lesson 17 — CSRs, traps and the M extension 67
Appendix A — Numbers that matter 73
Appendix B — Verilog style rules 76
Appendix C — Where to go next 79

PART I

The big picture

What an FPGA is, what the fabric is made of, and what "what it cost" means — the pages the rest of the book assumes. If you have written gateware for a living, start at Part II.

The big picture	A chip that becomes your circuit, and why Verilog is a description rather than a program
The fabric, and what it costs	Lookup tables, carry chains, the board, the chip, and the shape of the course

The big picture

This is a hands-on refresher in digital design, worked from a single logic gate to a 32-bit RISC-V processor running real programs on real silicon. It is written for someone who studied this once — perhaps a long time ago — and wants the ideas back *in their hands* rather than back in their head. So concepts are stated briefly and then built: simulated, inspected in a waveform viewer, synthesised, measured, and run on a board.

It assumes you know what a gate, a register and a clock are. It does not assume you remember what a blocking assignment does to a shift register, or why your counter synthesised into a latch.

Everything was built and measured on a **Radiona ULX3S** carrying a Lattice ECP5 `LFE5U-85F`, and every number in this book came out of the tools on that part. Roughly half the course is board-independent — gates, multiplexers, state machines, the ALU, the register file, instruction decode would be identical on any FPGA. Where a result depends on this particular fabric, and several of the most interesting ones do, the book says so.

Installing the toolchain, the `make` verbs, programming the board and the exact commands behind every number are in the companion volume, *Tools and Workflow*. This book is the concepts.

A chip that becomes your circuit

A microcontroller arrives with its hardware already decided. Somebody chose how many adders it has, how wide its registers are, which peripherals sit on which pins, and in what order it will do things. You get flexibility by changing the *sequence of instructions* it fetches; the silicon is fixed, and one instruction happens at a time.

An FPGA arrives with almost nothing decided. It is a field of uncommitted logic, flip-flops and wires, plus a very large number of switches deciding which wire connects to which. Configuring it does not give the chip a program to follow — it **builds a circuit out of the fabric**. That is the whole of the acronym: a *field-programmable gate array*.

The consequence never stops mattering. If your design needs thirty-two adders working at the same time, it has thirty-two adders, all adding on every clock edge. Nothing is shared unless you write the sharing, and nothing waits its turn unless you build the queue.

Programming it means loading configuration bits

There is no compiler emitting instructions here, because there is nothing to execute them. The toolchain produces a **bitstream**: a file of configuration bits that fills in every lookup table's contents and sets every routing switch in the part.

A bitstream therefore describes the whole chip, not your design. Lesson 00′s blinky — a counter and eight LEDs — produces a 274 KB bitstream on this part. Lesson 13′s complete RISC-V CPU, something like a hundred times the logic, produces 295 KB. Both configure the same fabric; most of either one says "unused".

On the ECP5 that configuration lives in on-chip SRAM, so it is volatile: `make prog` loads it for this power cycle, `make flash` writes the SPI flash the FPGA reloads itself from. An FPGA that has not been configured does nothing whatsoever. It is not running an idle loop; there is no loop.

Verilog is a description, not a program

This is the one real mental shift, and it is worth being blunt about.

There are no lines that "run". `assign y = a & b;` is not a statement executed when control reaches it — it is a piece of wiring that holds continuously, for as long as the chip is powered. `always @(posedge clk)` is not a loop; it is a register that updates on every rising edge, forever, whether or not anything else is paying attention. Two `always` blocks are not two threads taking turns. They are two pieces of hardware, both of them live, always.

A `for` loop in synthesisable code does not iterate in time either. It stamps out copies in space: thirty-two iterations become thirty-two copies of the body, all working at once. Loop count is area, not duration.

Two tools read the same file with very different strictness. The simulator accepts a great deal that no hardware can be — delays, `$display`, unbounded integers — because its job is to model behaviour. The synthesiser accepts a subset, and within that subset it quietly builds what you *described* rather than what you meant: an incompletely specified `if` becomes a latch, a missing `else` becomes a memory you did not ask for. Nothing warns you, because from the tool's point of view you asked for it.

Hence the order of work in every lesson: simulate first, then synthesise, then look at the schematic the synthesiser produced and check it is the circuit you had in mind.

Two ways to do it

Every lesson up to 09 can be done by typing Verilog, or by wiring blocks together visually in Icestudio, and you can switch at any lesson. The two cannot drift apart, because the Icestudio project is *generated* from the lesson's real Verilog — the diagram you wire up and the file you would type are the same design, laid out differently. Going the other way, `make graph` runs your Verilog through the synthesiser and hands you back a schematic.

Beyond lesson 09 the lessons are editor-only, and that is itself a lesson. Schematic entry stops scaling at roughly the complexity of a 32-bit datapath: a 32-bit ALU drawn as blocks is a wall of identical slices nobody can read, while as text it is a dozen lines with `[31:0]` on them. The industry made the same transition in the late 1980s for the same reason.

What this course is not

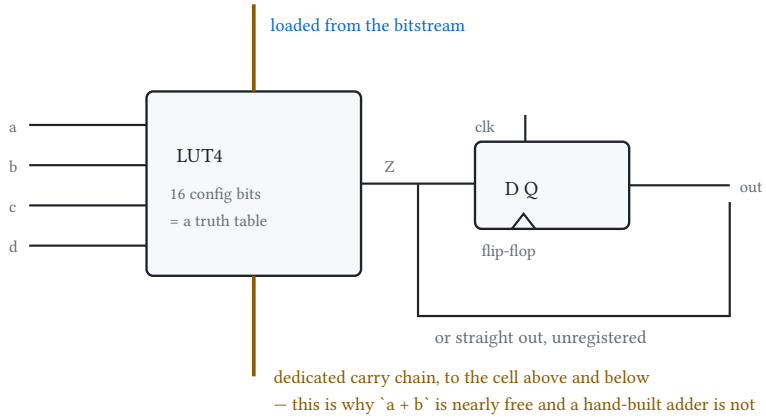
It is not a Verilog language reference, and it is not a complete treatment of computer architecture. Both exist and are better at being themselves; several are listed at the end of this book.

It is also not a tour of IP cores. Nothing here is instantiated from a vendor library. Every component — the adder, the register file, the UART, the CPU — is built and tested in a lesson, because the point is to have built it.

The fabric, and what it costs

What is actually inside

This is the single most useful piece of theory in the book, because almost every cost surprise in the course follows from it.



Every logic cell has a flip-flop whether the design uses it or not, which is why flip-flops are not the scarce resource.
Any function of up to four inputs costs exactly one LUT4, however complicated: a 2-input AND and a 4-input XOR are the same price.

An ECP5 logic cell. The LUT's sixteen configuration bits come from the bitstream, which is what "programming" an FPGA means.

An FPGA contains no gates. It contains **lookup tables**: sixteen configuration bits holding the truth table of whatever four-input function you asked for, selected by the four inputs. Your `a & b` does not become an AND gate; it becomes a LUT4 loaded with the AND truth table and two inputs unused.

Three consequences run through the whole course:

1. **Any function of up to four inputs costs exactly one LUT4**, however complicated. A 2-input AND and a 4-input XOR are the same price (lesson 01). Minimising gate count — half of a classic logic design course — mostly does not apply here.
2. **Flip-flops are not scarce.** Every logic cell has one whether the design uses it or not. This is why one-hot state encoding is *supposed* to be free on an FPGA — and lesson 05 measures a case where it is four times worse anyway.
3. **Some things have dedicated hardware and some do not.** A carry chain runs up each column of cells, so addition is nearly free: a 32-bit adder costs 17 carry cells and no LUTs. There is no dedicated shift hardware, so a 32-bit barrel shifter costs about 667 LUT4 — which is why lesson 10 finds that the shifter, not the adder, is 80% of the ALU.

Which operations are expensive is not intuition. It is a property of the fabric in front of you, and on a chip without a carry chain the answer would invert.

What "what it cost" means

Every lesson ends by asking what the design cost, and the phrase has one meaning here: **how much of the FPGA it used**. Not money, not runtime. `make synth` prints a count of the chip's own primitives, and these are the lines worth reading:

in the report	what it is
`LUT4`	four-input lookup tables — general-purpose logic, and usually the headline number
`TRELLIS_FF`	flip-flops: one bit of state each. Lesson 05′s design reports 113 of them, and the lesson accounts for all 113
`CCU2C`	carry cells, two bits of arithmetic each — the dedicated adder hardware, which is why `+` is so much cheaper than logic you build yourself
`PFUMX`, `L6MUX21`	multiplexers that glue LUT4s together into wider functions, so a 5- or 6-input function costs a little more than one LUT4
`TRELLIS_DPR16X4`	distributed RAM: 16 words of 4 bits built out of a logic cell's lookup tables, which is how a small array gets stored when a block RAM would be wasteful
`DP16KD`	block RAMs, 18 Kbit each — lesson 08′s subject
`MULT18X18D`	hardware 18×18 multipliers. Seeing one appear when you did not ask for it is lesson 08′s most useful surprise

Area is what you spend. **Time is a separate thing, and it is a constraint rather than a price**: the longest path of logic between two flip-flops has to settle inside one clock period. Adding logic to that path does not make the design slower — the design has no choice about its clock, the board's crystal does — it makes it *stop working*, and `nextpnr` says so when `make bit` fails timing. That is lesson 04′s subject.

Both quantities come back in a second or two, which is the whole reason this course can measure rather than assert. And measurement repeatedly contradicts what everyone knows:

MEASURED four received rules that did not survive contact with the fabric

- "One-hot encoding suits FPGAs, since flip-flops are free." On this four-state machine it cost **41 LUT4 against 12** — nearly four times worse (lesson 05).
- "Mealy machines need fewer states, so they are cheaper." Fewer states, yes — 3 against 4 — and **more logic**: 5 LUT4 against 2 (lesson 05).
- "The synthesiser will reduce a constant multiply to shifts." It spent **two hardware multiplier blocks** instead (lesson 08).
- "Build it from gates to understand it, then it is the same thing." The hand-built 32-bit adder cost **212 cells to `+`'s 17** (lesson 02).

None of those are here because they were read somewhere. They are here because `make synth` said so, and they are reproducible on your machine in the time it takes to read the paragraph.

The board

The ULX3S is an open-hardware board from Radiona, the makerspace in Zagreb, designed for teaching — the peripherals are the ones you want when learning, placed where you can reach them, with no mezzanine cards to buy. Four of them matter for this course: a **25 MHz** crystal clock, **8 LEDs** (active high), **7 buttons** (undebounced, which lesson 03 turns into a virtue) and **4 slide switches**, plus an **SSD1331 OLED** — 96×64, 16-bit colour, 4-wire SPI — for lessons 06 and 07, and an **FT231X** that is both the JTAG programmer and the serial port for lesson 09 onward. The SDRAM, SD card and video output are where you go after the course ends; the full pinout is in *Tools and Workflow*.

The chip

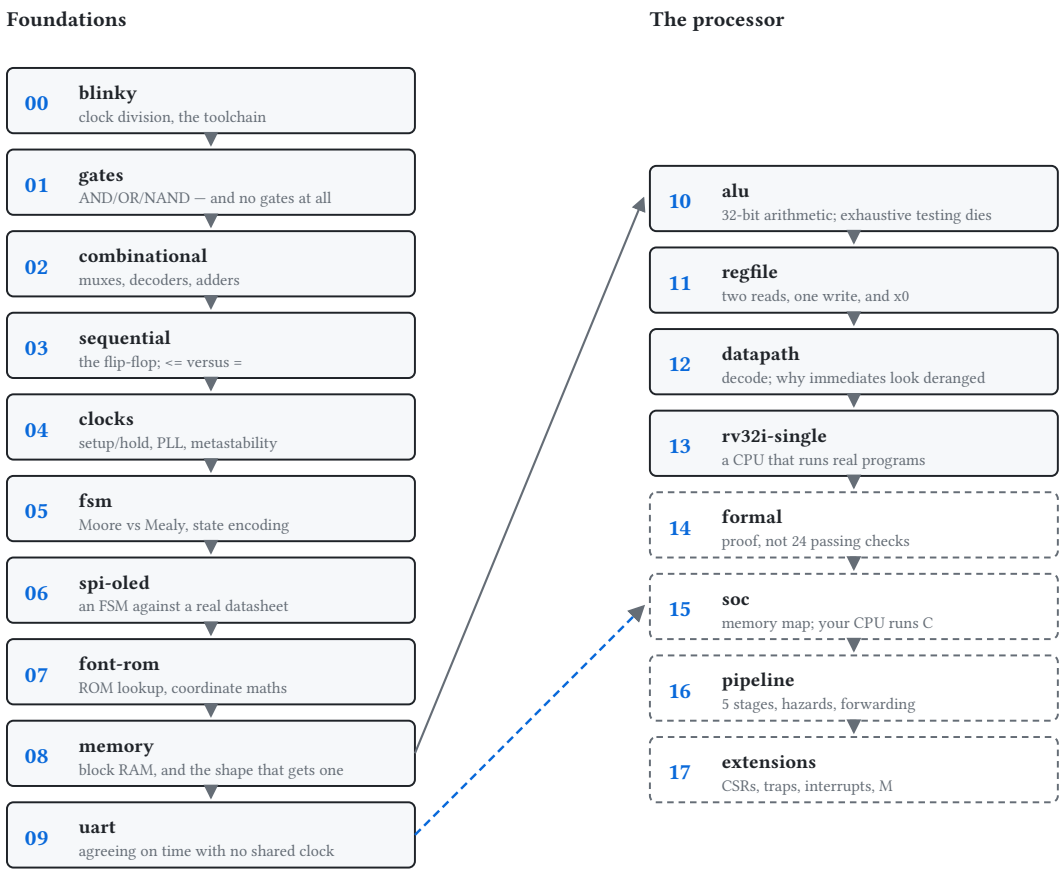
resource	count	what it is
LUT4	83,640	4-input lookup tables — the general-purpose logic
Flip-flops	83,640	one per logic cell, whether you use it or not
Block RAM	208 × 18 Kbit	about 3.7 Mbit of dedicated memory
Multipliers	156	18×18 hardware multipliers
PLLs	4	frequency synthesis, lesson 04

That is an `LFE5U-85F-6BG381C`: an ECP5 without SERDES, size grade 85 (about 84,000 logic cells), **speed grade 6** of three — a faster part would close timing at a higher clock with identical Verilog — in a 381-ball caBGA, commercial temperature range.

For calibration: lesson 13's whole RISC-V CPU uses about 1,765 LUT4, a little over **2%** of the part. Logic capacity will not be your constraint in this course, or in most projects.

The shape of the course

Eighteen lessons, each self-contained, each ending in something you can see working. The sequence is not arbitrary: every lesson exists because a later one needs it. The register file in lesson 11 is the CPU's register file; the UART in lesson 09 is how the CPU will say hello.



The course. Solid arrows are prerequisites; the right-hand column is the processor, assembled from parts built in the left-hand ones.

The five parts of this book follow that structure, and Part IV — the processor — depends on 00–05, 08 and 10–12. Lessons 06 and 07 are the display path and 09 is the serial path; the UART returns in lesson 15 as the processor's console, but the CPU in lesson 13 does not depend on either. If your goal is the processor and nothing else, you can read those three afterwards without being stuck.

PART II

Digital logic

Gates, combinational structures, clocked state, timing, and state machines. Ends with a synchronous controller you designed, simulated and ran on the board.

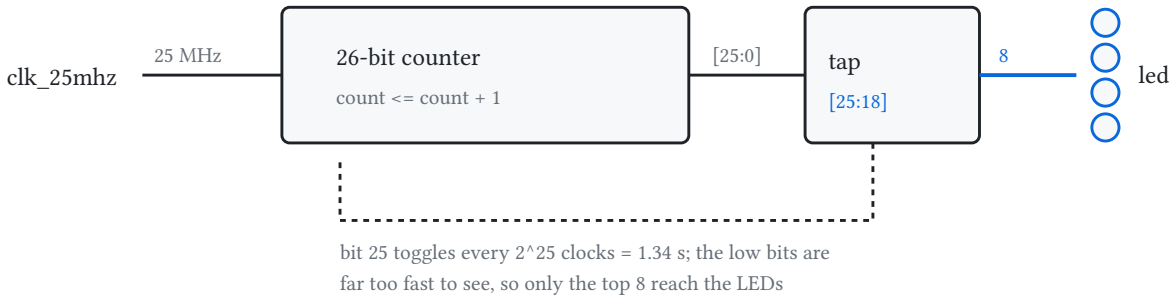
Lesson 00 — Blinky	A counter is a frequency divider, and every flip-flop is accounted for
Lesson 01 — Logic gates	What a LUT4 costs, and why minimising gate count stops mattering
Lesson 02 — Combinational logic	Multiplexers, decoders, and a hand-built adder against `+`
Lesson 03 — Sequential logic	Flip-flops, shift registers, debouncing, and `<=` against `=`
Lesson 04 — Clocks and timing	The critical path, a PLL, and crossing between clock domains
Lesson 05 — Finite state machines	The three-part form, Moore against Mealy, and encodings measured

Lesson 00 — Blinky

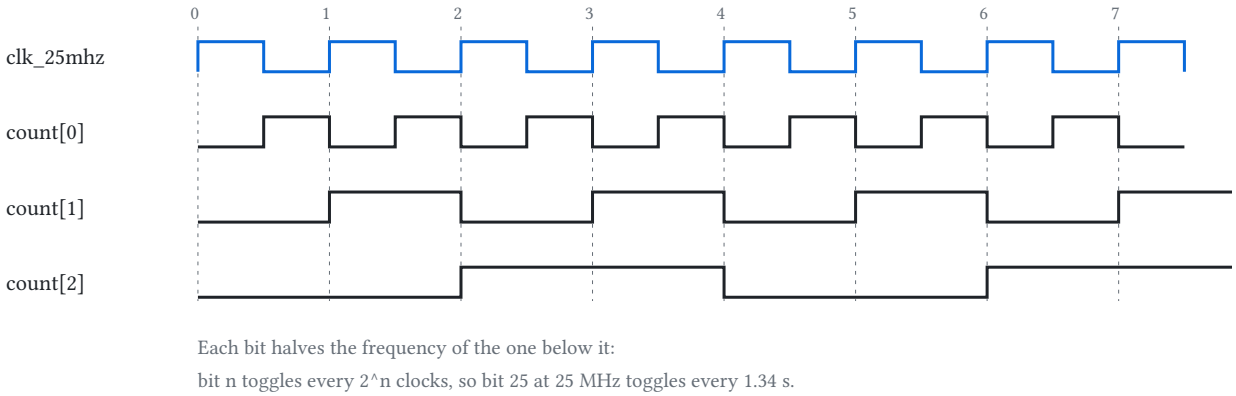
The idea

The board's crystal runs at 25 MHz. An LED toggling at that rate looks like an LED that is simply on, so to see anything the clock must be divided by tens of millions.

The cheapest divider in digital design is a binary counter. Add one to a register every clock and bit n toggles every 2^n clocks: bit 0 at 12.5 MHz, bit 1 at 6.25 MHz, and so on. Nothing is **built** to divide — the division is what counting already is, read one bit at a time.



Tapping the top eight bits of a 26-bit counter. At 25 MHz, bit 25 toggles every 1.34 seconds.



Each bit halves the frequency of the one below it — the whole of clock division, in one picture.

Theory

Non-blocking assignment

```
always @(posedge clk)
    count ⩽ count + 1'b1;
```

⩽ is non-blocking: every right-hand side in the block is evaluated before any left-hand side updates. That is what a bank of real flip-flops does — all of them sample their inputs at the same instant. Lesson 03 measures what happens when you use = instead, and the answer is two flip-flops fewer than you asked for.

Parameterised time constants

The testbench overrides `WIDTH` from 26 to 8, so the simulation finishes in milliseconds rather than simulating 67 million clock cycles. **Making a design's time constant adjustable is what makes slow hardware testable**, and it is a habit every later lesson depends on — lesson 03's debouncer and lesson 09's baud generator both do it.

Part-select

```
assign led = count[WIDTH-1 -: 8];
```

"Eight bits, counting downward from `WIDTH-1`." Writing `count[25:18]` means the same thing until someone changes `WIDTH`, at which point it silently reads out of range.

What it cost

MEASURED `blink` — a 26-bit counter
`make synth`

resource	count
LUT4	1
TRELLIS_FF	26
CCU2C	13

1 of the 85F's 83,640 LUT4 — less than 0.01% of the part.

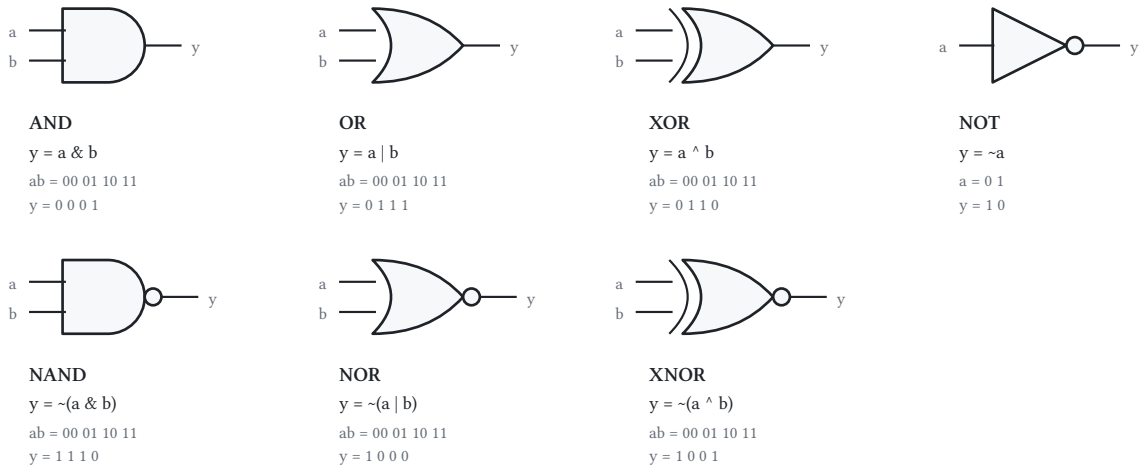
Twenty-six flip-flops: one per counter bit, exactly as expected. Thirteen carry cells do all 26 bits of the addition in dedicated hardware — two bits per cell — which is why a wide counter is nearly free here. General-purpose logic contributes a single LUT4.

The number is not "about thirty cells"; it is 26 flip-flops and 13 carry cells, and both are explainable. When a resource report contains something you cannot explain, you have inferred something you did not intend.

Lesson 01 — Logic gates

The idea

Worth knowing by sight: every schematic in this course, and every diagram `make graph` draws, is made of these.



A bubble on the output means "and then invert".

Each is a function of at most two inputs, so each costs exactly one LUT4 — a 2-input AND and a 4-input XOR are the same price.

The seven gates every digital circuit is described with. The bubble means “and then invert”.

A gate is a Boolean function of its inputs. In Verilog you write it as a permanent connection rather than as something that happens:

```
assign y_and = a & b;
```

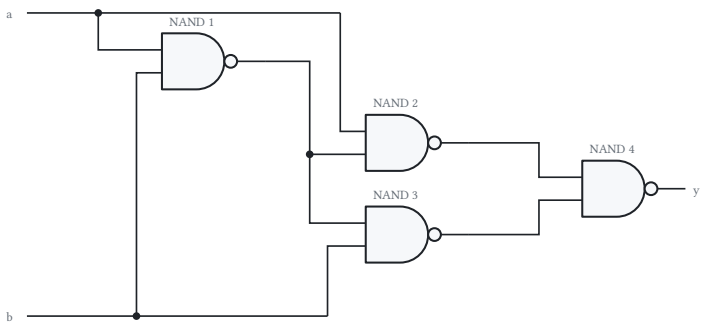
`assign` is not a statement that runs. There is no “when”. It is a wire: if `a` or `b` changes, `y_and` follows as fast as the silicon allows.

Theory

NAND is universal

Every Boolean function can be built from NAND alone:

function	from NAND	why
`NOT a`	`NAND(a, a)`	both inputs equal, so the AND is just `a`
`a AND b`	`NAND(NAND(a,b), NAND(a,b))`	the second NAND is a NOT
`a OR b`	`NAND(NOT a, NOT b)`	De Morgan: `a|b == ~(~a & ~b)`
`a XOR b`	four NANDs	the classic construction



The classic one, drawn out: exclusive-OR from four NANDs. Hold on to this picture — the measurement below is about what became of it.

Historically this mattered enormously: one gate type meant one mask, one part number, one bin of 7400s on the shelf.

Exhaustive testing

A two-input gate has four possible inputs. There is no need to **choose** test cases — check all of them, against truth tables written out by hand rather than computed with the same operator being tested.

This is the last lesson where exhaustive testing is this trivial, and lesson 02 the last where it is possible at all: the counts go 4, then 512, then — by lesson 10 — 4.6×10^{19}. Build the habit while it is cheap.

What it cost, and why it is the point

MEASURED `gates_top` — eleven outputs, no state at all

`make synth`

resource	count
LUT4	7

7 of the 85F's 83,640 LUT4 — less than 0.01% of the part.

No flip-flops at all: this design has no memory.

MEASURED eleven outputs from seven LUT4s

Fewer LUTs than outputs, because the synthesiser does not implement what you **wrote**: it works out what each output *is* as a function of `a` and `b`, and shares whatever can be shared.

The NAND-only constructions are the clearest case. `n_xor` was written as a chain of four NANDs; the synthesiser evaluated the function, found XOR, and programmed one lookup table. **It never built the chain.**

The fabric has no gates, only 4-input lookup tables, so a 2-input AND and a 4-input XOR cost exactly the same — one LUT4 each, as does any other function of up to four inputs however complicated. Minimising gate count, half of a classic digital design course, is therefore almost pointless here. What costs is the number of LUTs, their *depth* — how many in series, which sets your clock speed — and the routing between them.

The boundary that does matter

A LUT4 holds any function of four inputs. Five inputs needs at least two LUTs, no matter how simple the function:

function	cost
`(a & b) | (c ^ d)`	1 LUT4
`((a^b) & (c|~d)) | (~a&b&c&d) | (a&~b&~c)` — far uglier, same 4 inputs	1 LUT4
`(a & b) | (c ^ d) | (e & ~a)` — one more input	2 LUT4 + 1 mux

Count inputs, not operators. "Simplifying" a four-input expression from five operators to three saves nothing.

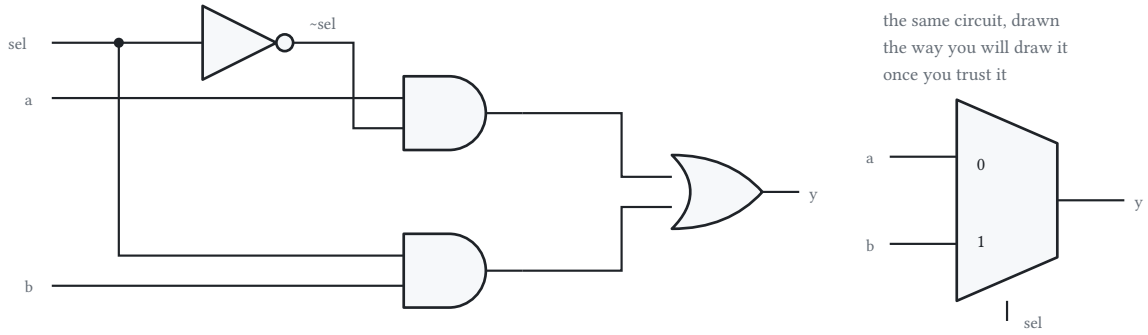
Lesson 02 — Combinational logic

The idea

Real designs are not built from gates any more than prose is built from letters. Three structures account for most of a processor.

The multiplexer selects: one of several inputs reaches the output. Every point in a datapath where a value "comes from here, or from there" is a mux, and lesson 13's CPU is mostly muxes with arithmetic between them.

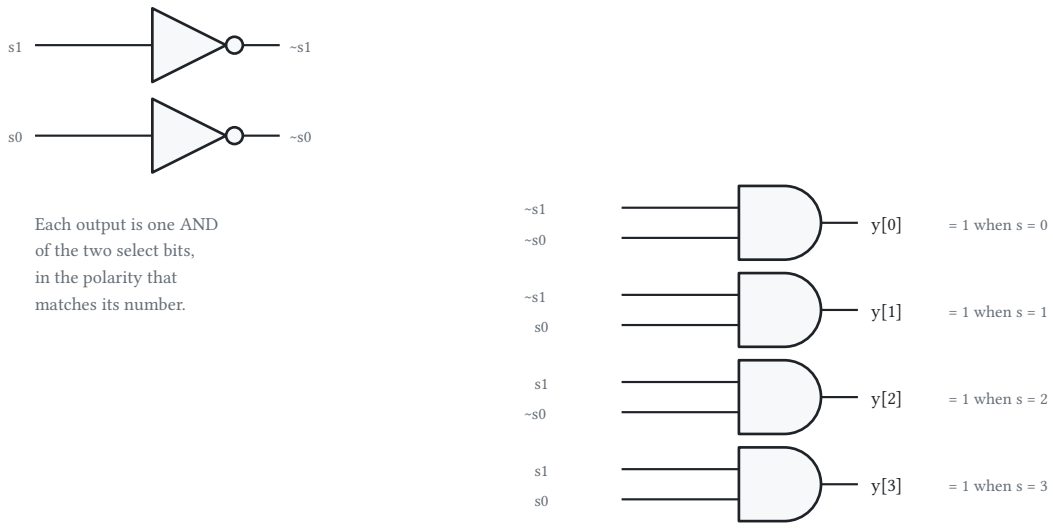
The decoder is the mux's mirror image: one input number, and exactly one of 2^n outputs goes high. Wherever a number must become a *choice* — instruction decode, address decode, one-hot state — there is a decoder.



y = (a & ~sel) | (b & sel) — three inputs, one function, and therefore one LUT4.

What the gates make obvious and the symbol hides: both inputs are computed, every cycle.
A multiplexer does not choose which path to evaluate. It evaluates both and throws one away.

A 2:1 multiplexer, in gates and as the symbol you will use once you trust it. Both inputs are computed every cycle; one is thrown away.



A decoder is a multiplexer in a mirror: one line raised out of many, rather than one of many inputs selected onto a single output.

A 2-to-4 decoder: one AND per output, each taking the polarity of the select bits that matches its own number.

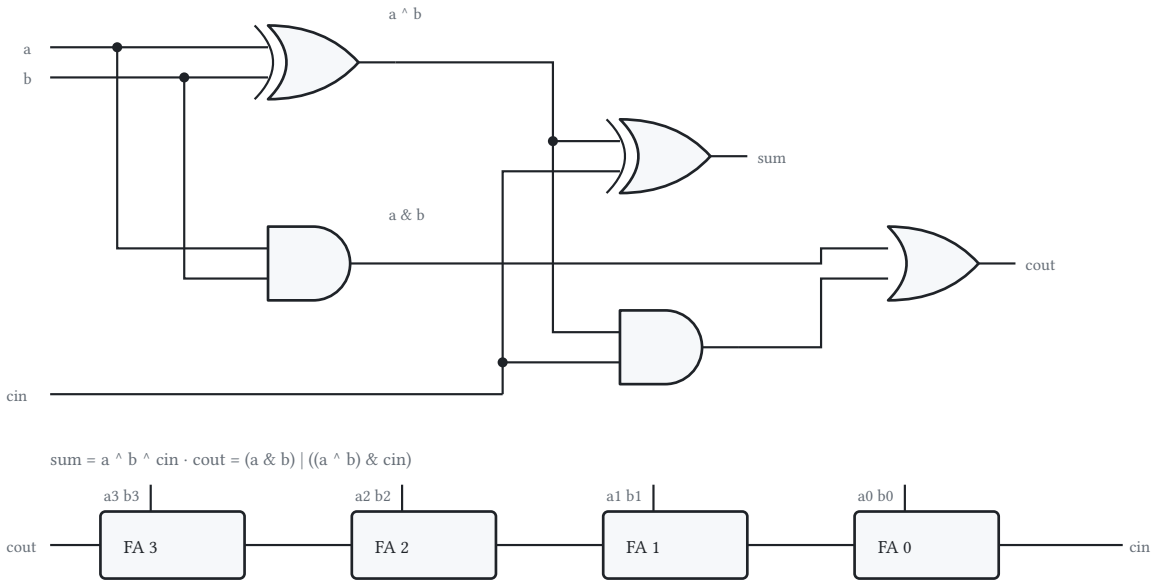
The adder is the first structure with an interesting internal shape, because each bit's result depends on a carry from the bit below.

Theory

Carry propagation is why addition takes time

One bit of addition is two gates' worth of logic:

```
assign s    = a ^ b ^ cin;                  // odd number of ones
assign cout = (a & b) | (cin & (a ^ b));    // at least two ones
```



One bit of addition in five gates, and four of them chained. The carry is the dependency that makes addition take time.

Chain *W* of them and the top bit of the sum cannot settle until the carry has propagated through every stage below it. At 4 bits nobody cares. At 32 bits it is the slowest thing in the CPU — and lesson 10 measures exactly that.

Two width traps

Both are silent, and both come from the same root: **in Verilog, width is part of the meaning.**

Watch out. Shifts. The result of `a << b` is as wide as `a`. So `assign y = 1'b1 << sel;` shifts a **one-bit** value, everything falls off the end, and `y` is always zero. The literal must be widened before it is shifted.

Carries. `a + b + cin` assigned to a W-bit destination throws the carry away. Widening the intermediate to W+1 bits is what keeps it. At 4 bits this is wrong in exactly half of the 512 possible cases — and the sum bits are never corrupted, only the carry, which is what makes it hard to spot.

Latches

A combinational `always` block that fails to assign its output on some path infers a **latch** — a storage element with no clock, which is almost never what you meant. The habit is to assign a default first and then override:

```
always @(*) begin
    y = 1'b0;            // default FIRST, so no path is uncovered
    case (sel) ... endcase
end
```

What it cost

MEASURED `comb_top` — the whole design

`make synth`

resource	count
LUT4	14
CCU2C	3
PFUMX	2

14 of the 85F's 83,640 LUT4 — 0.02% of the part.

Small, because almost everything in this lesson is a function of four or fewer inputs. The three carry cells are the 4-bit adder the board demo uses.

Two comparisons inside it are the lesson, though, and both are worth reproducing module by module.

MEASURED `seg7` — a 16-way `case` statement

`make synth M=seg7`

resource	count
LUT4	7

MEASURED `seg7_rom` — the same table as an array of constants

`make synth M=seg7_rom`

resource	count
LUT4	7

Identical. The synthesiser cannot tell the difference, because there is none. A `case` is not "logic" and an array is not "memory"; both describe one function of four inputs, and only the function survives.

MEASURED `adder_ripple` — 32 bits built from full adders

`make synth M=adder_ripple`

resource	count
LUT4	138
PFUMX	44
L6MUX21	30

MEASURED `adder` — the same 32 bits, written `a + b + cin`

`make synth M=adder`

resource	count
CCU2C	17

212 cells against 17, a factor of twelve. At 4 bits — `P="W 4"` on either command — it is 24 LUT4 and 17 muxes against 3 carry cells, so the gap widens with width rather than closing.

The reason is what is *absent* from the hand-built rows: there are no `CCU2C` cells at all. The carry chain is dedicated hardware, one cell per two bits, and when you write `a + b` the synthesiser recognises addition and maps straight onto it. Write the same function as explicit gates and it sees gate soup — it cannot prove that pile of XORs and ANDs is an adder, so the hard carry logic goes unused.

Lessons 01 and 02 point in opposite directions and both are true. In lesson 01 the synthesiser *discarded* your structure and kept only the function. Here it *kept* your structure, because your structure prevented it from recognising the function.

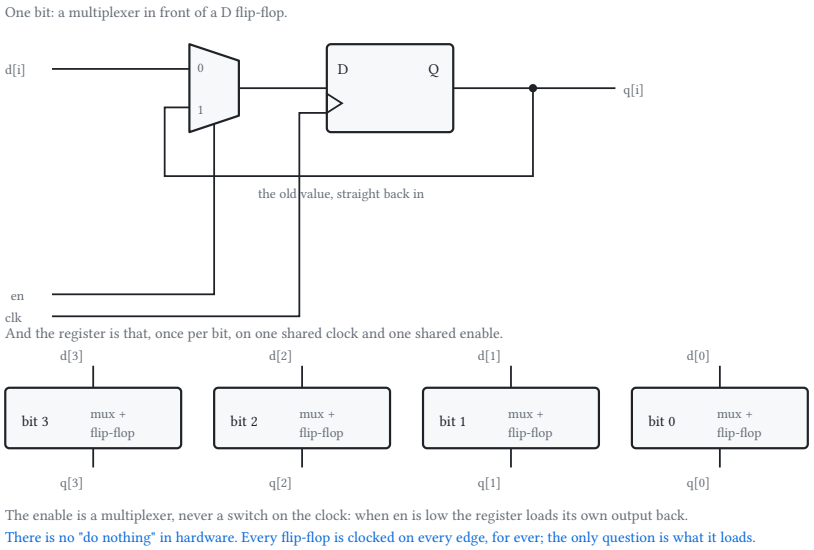
The rule underneath both: **write what you mean, at the highest level you can, and let the tool match it to the hardware.** Build the ripple adder once to understand what a carry is. Then write `+` forever.

Lesson 03 — Sequential logic

The idea

Everything so far had no memory. A D flip-flop changes that: on each rising clock edge it samples its input and holds that value until the next edge.

Nothing else in this course stores a bit. A register, a shift register, a counter, a state machine and a 32-bit CPU are all arrangements of flip-flops with combinational logic deciding what goes into them.



A register, drawn out: one flip-flop per bit, and a multiplexer in front of each deciding whether to take new data or reload the old value.

Theory

⩽ versus =, as two circuits

The same three-stage shift register, written twice:

```
always @(posedge clk) begin
    a ⩽ d;
    b ⩽ a;   // the OLD a
    c ⩽ b;   // the OLD b
end
```

```
always @(posedge clk) begin
    a = d;
    b = a;   // the NEW a
    c = b;   // the NEW b
end
```

Non-blocking evaluates every right-hand side before updating any left-hand side, so each stage samples what its neighbour held at the start of the edge. A pulse walks along: stage `a` after one clock, `b` after two, `c` after three.

Blocking takes effect immediately, so each line sees the result of the line above. All three registers load the same value on the same edge. The delay line is gone.

MEASURED the bug, as a number

`shift_nonblocking`	**3** flip-flops
`shift_blocking`	**1** flip-flop

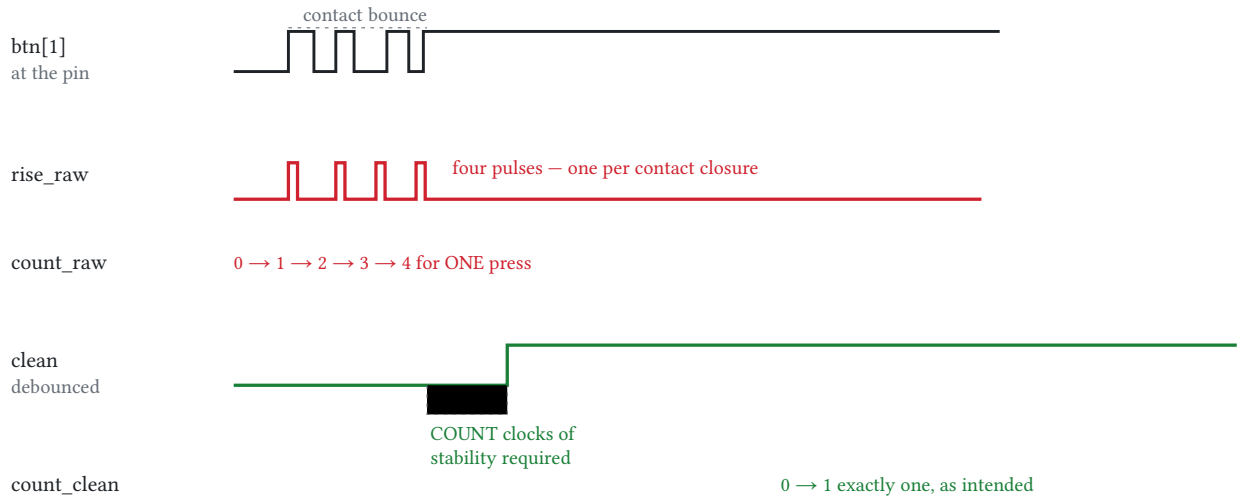
One flip-flop. The synthesiser worked out that `b` and `c` always equal `a`, built one register and wired its output three ways.

Watch out. Nothing warns. It compiles, it simulates, it synthesises, and `make lint` is quiet. The three-stage delay you asked for became a one-stage delay, and the only ways to notice are to know the rule or to count the flip-flops. This is the most common serious mistake in Verilog.

A switch is not a digital input

A button is two pieces of springy metal. When they meet they bounce apart and meet again — several times, over roughly 1–10 ms.

At 25 MHz, 10 ms is **250,000 clock cycles**. The bounce is not a blur or a glitch to the FPGA: it is tens of thousands of clocks of perfectly clean, fully sampled, entirely real transitions. **One press genuinely is several presses**, as far as the hardware can tell.



One press. The raw path counts three times; the debounced path counts once.

Lesson 02 did not care — combinational gates just flicker, and a flickering LED is not a wrong answer. The moment an edge is used to *count*, every bounce counts.

The defence is a saturating counter: the input must disagree with the current output *continuously* for `COUNT` clocks before the output follows. Any shorter disagreement resets the timer, which rejects bounce of any pattern rather than only bounce shorter than one sample period. The price is latency, so you pick the smallest `COUNT` that comfortably exceeds the switch's bounce time.

The apparent contradiction

```
module reg_w #(parameter W = 32)(input clk, en, ...);
    always @(posedge clk)
        if (en) q <= d;      // no else
endmodule
```

In lesson 02 an `always @(*)` block without an assignment on every path inferred a latch, and that was a bug. Here the identical-looking omission is **correct**.

The difference is the clock. A flip-flop already holds its value between edges, so "assign nothing" is a real, cheap, well-defined behaviour — it is just the flop's enable input. In a combinational block there is no storage to fall back on, so the tool has to invent some. Same syntax, opposite meaning, decided entirely by whether the block is sensitive to a clock edge or to its inputs.

What it cost

MEASURED `seq_top` — the whole design

`make synth`

resource	count
LUT4	24
TRELLIS_FF	29
CCU2C	9
PFUMX	4
L6MUX21	1

24 of the 85F's 83,640 LUT4 — 0.03% of the part.

The nine carry cells are the debounce counter's 18-bit increment — one per two bits, exactly as lesson 02 predicted.

MEASURED every one of the 29 flip-flops accounted for

debounce: an 18-bit counter plus `clean`	19
two edge detectors, one `prev` each	2
two 4-bit counters	8
total	**29**

Lesson 04 — Clocks and timing

The idea

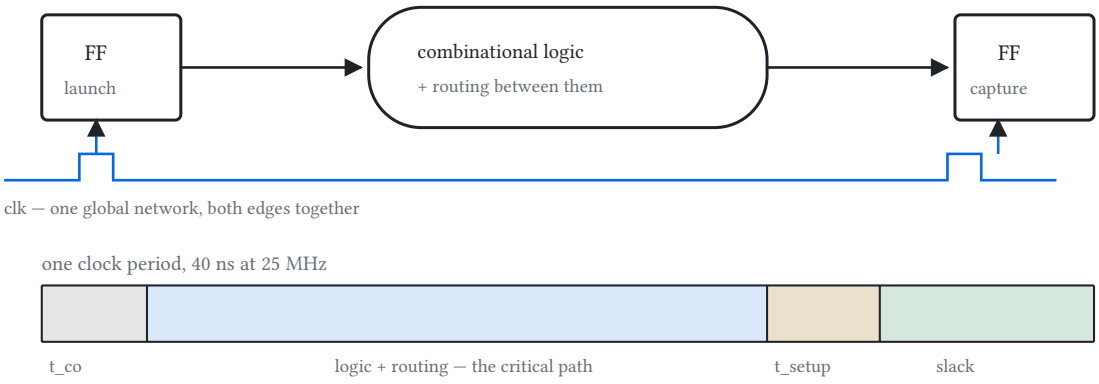
Three lessons used `clk_25mhz` as if it were simply given. It is not.

Setup, hold, and the critical path

A flip-flop needs its data input stable *before* the clock edge (setup) and *after* it (hold). Between any two flip-flops there is a budget: one clock period, minus the first flop's output delay, minus the second's setup time. Everything else is available for logic and routing.

The longest such path in the design sets the maximum frequency:

$$\text{max frequency} = \frac{1}{\text{critical path delay}}$$



The longest path sets the maximum frequency. Slack below zero is not a slow design — it is a broken one.

One clock period, as a budget. Everything the data must do between two flip-flops has to fit inside it, and what is left over is the slack nextpnr reports.

So **a design is not correct or incorrect in isolation — it is correct at a frequency.** The same Verilog that works at 25 MHz may fail at 100, and nothing in the source says which. This is also the real argument for pipelining, twelve lessons early: lesson 16 splits the CPU into five stages not for elegance but because a shorter critical path permits a faster clock.

Enables, not gated clocks

Lesson 00 divided the clock with a counter and used the result as *data*. Using it as a *clock* is the mistake worth naming:

```
wire tick = (div == LIMIT);                          // yes: an enable
always @(posedge clk_25mhz) if (tick) count <= count + 1'b1;

always @(posedge div[15]) count <= count + 1'b1;    // no: a derived clock
```

An FPGA has a small number of **global clock networks** — dedicated low-skew routing that reaches every flip-flop at very nearly the same instant. A signal generated in logic travels on ordinary routing, arrives at different flops at measurably different times, and the timing analyser can no longer give a meaningful answer. You can watch the allocation happen in the build log:

```
Info:  promoting clock net clk_100mhz to global network
Info:  routing clock net $glbnet$clk_100mhz using global 1
```

The PLL is a hard block

The ECP5's `EHXPLLL` is a real analogue phase-locked loop in the corner of the die, not logic. Its parameters are computed by a calculator rather than chosen:

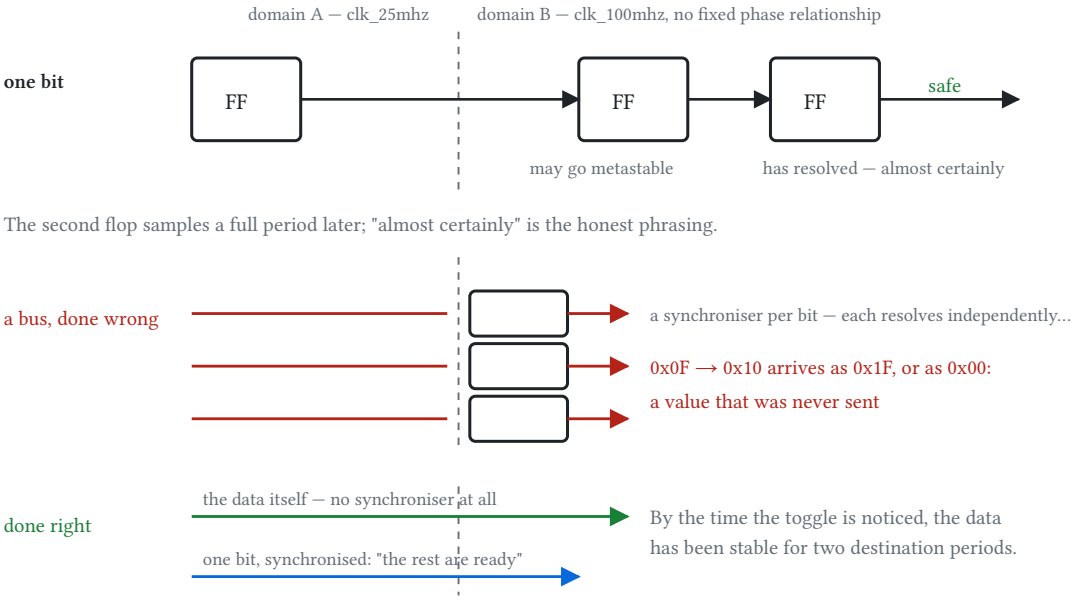
```
ecppll -i 25 -o 100 --clkout1 40
  → VCO 600 MHz, divisors 6 and 15
```

The VCO must land in its legal range (roughly 400–800 MHz) and **every output is that one VCO frequency divided by an integer**. You cannot ask for an arbitrary pair of frequencies.

Three consequences:

1. **It cannot be simulated.** Icarus has no model and Verilator has no definition. This is the first lesson where what you simulate and what you synthesise differ — the lesson's Makefile names a simulatable subset, and the PLL is verified structurally and on hardware instead.
2. **`locked` matters.** The loop takes tens of microseconds to settle. Logic in a PLL domain must be held in reset until then.
3. **PLLs are finite.** Four on this part, not one per module.

Crossing between domains



Two domains, three cases: a two-flop synchroniser for one bit, a synchroniser per bit corrupting a bus, and the handshake that fixes it.

Two clocks not derived from one another have **no fixed phase relationship**. Sooner or later a signal from domain A changes at exactly the wrong moment relative to domain B's edge — inside the setup/hold window. The receiving flip-flop then goes **metastable**: its output sits between 0 and 1 for an unbounded time before resolving arbitrarily.

Watch out. This cannot be simulated. Verilog has no notion of setup time, so the simulator always gives a clean, defined, repeatable answer no matter how badly the crossing is built. Simulation cannot show you this bug and cannot prove you have fixed it.

The defence is *structural*: you follow the rule because the rule is correct, not because a test went green. It is also why "it worked on my bench for an hour" means nothing — a mean time between failures of three weeks looks exactly like a working design on a Tuesday afternoon.

For **one bit**, a two-flop synchroniser. The first flop may go metastable; the second samples it a full period later, by which time it has almost certainly resolved. "Almost certainly" is the honest phrasing.

For **a bus**, a synchroniser per bit is a *bug*. Each bit resolves independently, so some arrive one clock and some the next, and the destination briefly sees a value that was never sent — `0x0F → 0x10` observed as `0x1F` or `0x00`. Instead, synchronise *one* bit and use it to say when the others are safe to read; the data itself crosses with no synchroniser at all, because by the time the request's toggle is noticed the data has been stable for two destination clock periods.

What it cost

MEASURED `clocks_top` — the whole design

`make synth`

resource	count
LUT4	19
TRELLIS_FF	67
CCU2C	26
EHXPLLL	1

19 of the 85F's 83,640 LUT4 — 0.02% of the part.

The design is almost all flip-flops and carry: two counters, the synchronisers and the handshake registers. The row that matters is the last one — **the PLL is one `EHXPLLL` and no logic at all**, because it is a hard block sitting in the corner of the die rather than something built out of the fabric. You cannot have five of them; the part has four.

What it cost in time

MEASURED timing on the real build

`clk_25mhz` domain	achieves 342.82 MHz — PASS at 25
`clk_100mhz` domain	achieves 278.09 MHz — PASS at 100
cross-domain path	1.49 ns, **found and reported separately**

nextpnr finds the crossing unaided, because it cannot analyse it as a normal path. An unexpected `→ posedge <other clock>` line in your report is the tool handing you a bug report.

MEASURED why 16× oversampling is not an option at 25 MHz

Lesson 09's UART wants to sample at 16 times the baud rate. But 115,200 × 16 = 1,843,200 Hz, and 25,000,000 / 1,843,200 = 13.5634 — not an integer. The nearest divisors give +4.33% and −3.12%, which is 41% and 30% of a bit adrift by the stop bit. The textbook technique is simply unavailable on this clock.

The honest part about forcing a failure

Two attempts to build something deliberately slow were defeated by the synthesiser. First a wrapper read only the low byte of the result, so the upper 24 bits were pruned and the measured frequency went *up* with chain depth. Then, with every bit live, a 16-deep chain still collapsed to 8 carry cells and 9 LUTs — the arithmetic was folded.

You cannot read your critical path off your source code. The thing that gets built is not the thing you wrote. Lessons 01 and 02 showed that for area; it is equally true for delay. Read the tool's report.

The reliable way to see a failure is to ask for more speed, not to build something slower:

MEASURED the same placed design, asked for different clocks

asked for	achieved	verdict
100 MHz	357.65 MHz	PASS
300 MHz	357.65 MHz	PASS
400 MHz	357.65 MHz	**FAIL**

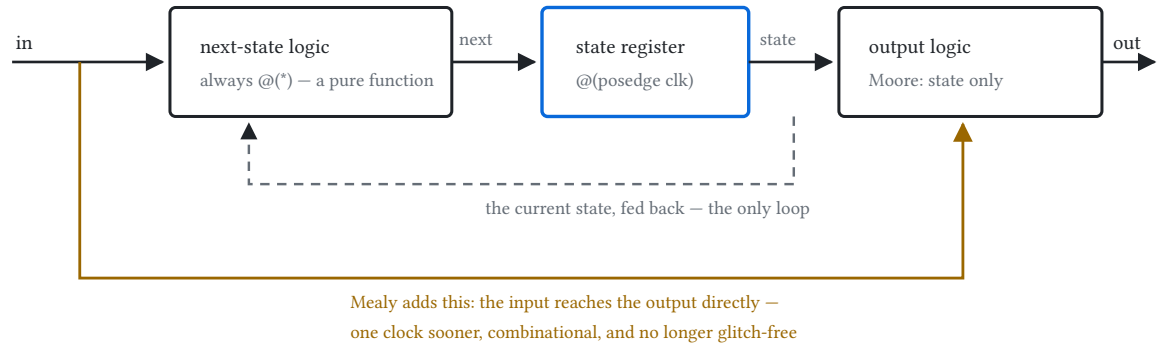
The achieved number never changes — it is a property of the placed and routed design. What changes is whether it is enough. That is what timing closure *is*.

Lesson 05 — Finite state machines

The idea

A state machine is a register holding *where you are*, and combinational logic deciding *where you go next*. That is all, and everything structural follows from it:

1. a **state register** — one clocked block, containing nothing else
2. **next-state logic** — one combinational block, a pure function
3. **output logic** — from state (Moore), or state and inputs (Mealy)



Splitting the register from the logic is not stylistic: it makes the only sequential element trivially obvious.

The canonical three parts, and where Moore and Mealy differ: one dashed feedback path, and one coloured path that is the whole of the distinction.

Splitting (1) from (2) is not stylistic. It makes the only sequential element trivially obvious, so the next-state block reads as a plain truth table and you never wonder what is registered.

The SPI engine in lesson 06, the UART in lesson 09 and the CPU's control in lesson 13 are all this one pattern at increasing scale. The CPU is not a different kind of thing; it is a state machine whose state includes a program counter.

Draw the diagram first, on paper, before any Verilog: bubbles for states, arrows labelled with the condition that takes them. Skipping that step is how you end up with a state you cannot reach and a transition you cannot explain.

Moore and Mealy

Two machines detecting the same sequence, differing only in where the output comes from:

	Moore	**Mealy**
output depends on	state	state **and** input
states needed	4	3
timing	one clock **late**	**immediate**
output is	registered, glitch-free	combinational

Neither is correct in general. The question is whether the consumer needs the answer *now* or needs it *clean*. A Mealy output feeding another FSM's next-state logic in the same cycle is a combinational path between two state machines — occasionally what you want, frequently a timing problem.

Two textbook rules, measured

MEASURED Mealy has fewer states and costs more

`seq_moore`	2 `LUT4`, 2 `TRELLIS_FF`
`seq_mealy`	**5 `LUT4`**, 2 `TRELLIS_FF`

Mealy genuinely needs fewer states — 3 against 4 — and the saving buys nothing, because both counts fit in two flip-flops, while making the output depend on the input adds real gates. The textbook trade is an *ASIC* trade, where you count flops.

MEASURED one-hot encoding, which is supposed to suit FPGAs, loses badly

encoding	cost
the synthesiser's own choice	11 `LUT4`, 3 `PFUMX`, 5 `TRELLIS_FF`
`"binary"`	12 `LUT4`, 3 `PFUMX`, 5 `TRELLIS_FF`
`"one-hot"`	**41 `LUT4`**, 18 `PFUMX`, 12 `L6MUX21`, 7 `TRELLIS_FF`

Nearly four times worse. Four states is too few for one-hot's decode saving to appear, while every comparison against the state must now be built from four bits instead of two. One-hot earns its keep on *wide* state machines, not narrow ones.

In Verilog the encoding is an attribute on the state register — not on the module, where it does nothing. Without it the synthesiser re-encodes as it sees fit, so the constants you wrote are a suggestion rather than an instruction.

`make synth` takes two seconds, and it outranks anything anyone tells you about what *usually* costs less.

The mistake everyone makes

A lock opening on the sequence **1, 2, 1, 3**. The states are easy. The interesting question is *what happens on a wrong symbol.*

The obvious answer — go back to the start — is wrong. Enter `1, 2, 1, 2`: after `1, 2, 1` the machine wants `3` and gets `2`. But the last two symbols it saw were `1, 2`, which is a valid *prefix* of the code. Restarting throws that away, and the lock then refuses `1, 2, 1, 2, 1, 3` even though the correct code is right there at the end.

Every state needs the same treatment: on a wrong symbol, fall back to the longest suffix of what you have seen that is still a prefix of the code. Doing it by hand is fiddly and easy to get subtly wrong — it is precisely what the Knuth–Morris–Pratt string-matching algorithm automates, and precisely the class of bug lesson 14's formal tools find without being asked.

Testing changes shape

An FSM's input is a *sequence*, so there is no space to sweep. The test is a set of chosen journeys through the state graph, and the discipline is **transition coverage**: has every arrow in your diagram been taken at least once?

The traffic-light test walks the full cycle for that reason, and then runs many clocks with no ticks to confirm the machine does not advance without its enable.

What it cost

MEASURED `fsm_top` — the whole design

`make synth`

resource	count
LUT4	75
TRELLIS_FF	113
CCU2C	49
PFUMX	12
L6MUX21	3

75 of the 85F's 83,640 LUT4 — 0.09% of the part.

MEASURED 113 flip-flops, and the proportion is the lesson

one-second tick divider (25 bits)	25
traffic light: 2 state + 3 timer	5
four debouncers	76
four edge detectors	4
lock state	3
total	**113**

The state machines are 8 flip-flops out of 113. The rest is debouncing and timekeeping — the plumbing needed to talk to the physical world. That ratio is normal, and worth seeing once: the interesting logic is usually a small fraction of a real design.

PART III

Interfaces and memory

A display over SPI, a font ROM, the FPGA's own memories, and a UART. Ends with a small interactive system with a screen and a serial console.

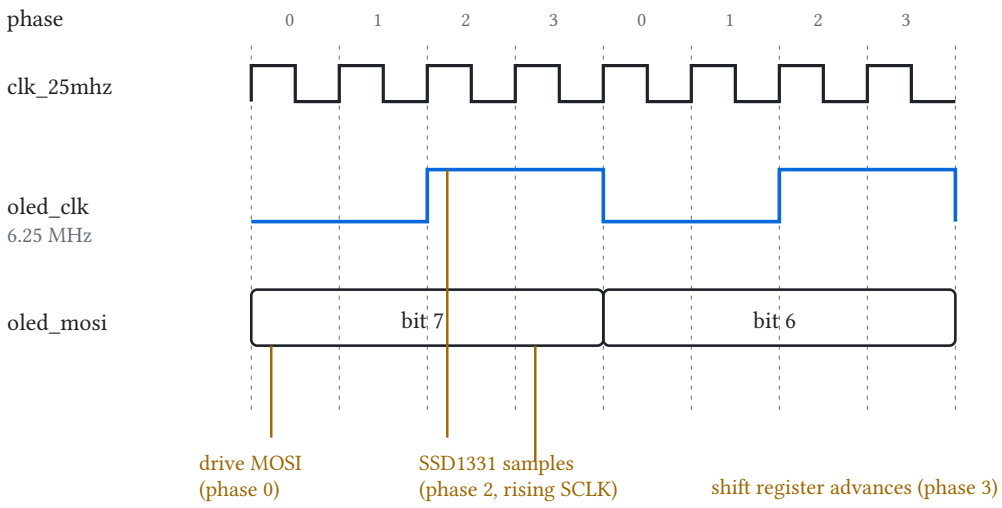
Lesson 06 — SPI and the OLED	A state machine pointed at a datasheet, and what its test cannot prove
Lesson 07 — Font ROM and coordinate maths	Choosing constants so the arithmetic disappears
Lesson 08 — Memories	The one line that decides between a block RAM and 919 LUT4
Lesson 09 — UART	Sampling, framing, and a testbench that asserted something false

Lesson 06 — SPI and the OLED

The idea

Everything so far was self-contained: your logic, your clock, your decisions. This is the first lesson where another piece of silicon has opinions, written down in a datasheet, and your design must satisfy them.

The SSD1331 speaks 4-wire SPI. The contract is: you supply the clock, at no more than about 6.6 MHz; data goes out most significant bit first; `D/C` is a *level* rather than a bit in the stream, saying whether this byte is a command or pixel data; and `RES#` must be held low on power-up and then released with a real delay before anything else.



Four system clocks per bit: data is set up two clocks before the chip samples it.

One byte. Four system clocks per bit gives 25 MHz ÷ 4 = 6.25 MHz, inside the datasheet's limit.

Theory

Set up the data, then clock it

The four phases do distinct jobs: drive the data line on phase 0, raise the clock on phase 2 so the data has been stable for two clocks before the chip samples it, and advance the shift register on phase 3.

"Set up the data, *then* clock it" is the whole of synchronous interface design. The same reasoning sets the sample point in lesson 09's UART receiver, where the receiver cannot control the clock and must infer the safest moment to look. Phase 1 in this design does nothing at all, and that idleness is the setup time.

The handshake

```
if (!busy && !start) begin ... end
```

`busy` is the entire interface between the bit-level engine and everything above it. The sequencer that pushes initialisation bytes and pixel data obeys one rule — do not touch `data` or raise `start` while `busy` is high — and therefore needs to know *nothing* about SPI timing.

That separation is why the same sequencer survives unchanged into lesson 07, which replaces only the pixel function.

Watch out. `start ⩽ 0;` sits at the top of the sequencer's always block, before the case. That makes `start` a one-clock pulse by default which any branch may override. Forget it and `start` stays high, the engine re-triggers immediately, and you send one byte twenty-five million times a second.

No framebuffer

96 × 64 × 16 bits is 98,304 bits, which would fit in block RAM easily. It is not needed, because the SSD1331's internal address pointer auto-increments: set the column and row window once during initialisation, and every subsequent data byte lands in the next pixel automatically.

So the design only has to produce bytes in raster order, and `x`/`y` counters mirroring the chip's own pointer are enough. The pixel is *computed* from the coordinate the moment it is asked for:

```
wire [15:0] pixel = { x[6:2], y[5:0], frame[7:3] };
```

Recognising that a buffer is unnecessary is worth as much as knowing how to build one. Lesson 08 builds the framebuffer that lesson 15's software will need — but a design that can compute its output on demand should.

What it cost

MEASURED `oled_top` — the whole display controller
`make synth`

resource	count
LUT4	108
TRELLIS_FF	72
CCU2C	22
PFUMX	29
L6MUX21	4

108 of the 85F's 83,640 LUT4 — 0.13% of the part.

The 72 flip-flops are the shift register, the phase and bit counters, the state, the reset delay counter, and the x/y/frame counters. The 29 multiplexers are mostly the 31-entry initialisation ROM, which the synthesiser built out of logic because it is far too small to be worth a block RAM — a decision lesson 08 examines properly.

Testing an interface

The testbench decodes the SPI stream the way the OLED would: it samples the data line on each rising clock edge, assembles bytes, checks the first 31 against the expected initialisation sequence with `D/C` low, then confirms pixel data keeps arriving with `D/C` high.

It reconstructs bytes *from the wire* rather than peeking at internal signals, so it tests the contract the other chip cares about. A bit-order or clock-phase error that an internal check would miss fails here.

But note the limit: a passing simulation does not prove the design meets a datasheet. Halve the clock divider and the simulation still passes while the hardware probably will not. When another chip's timing is involved, simulation is necessary and not sufficient.

Lesson 07 — Font ROM and coordinate maths

The idea

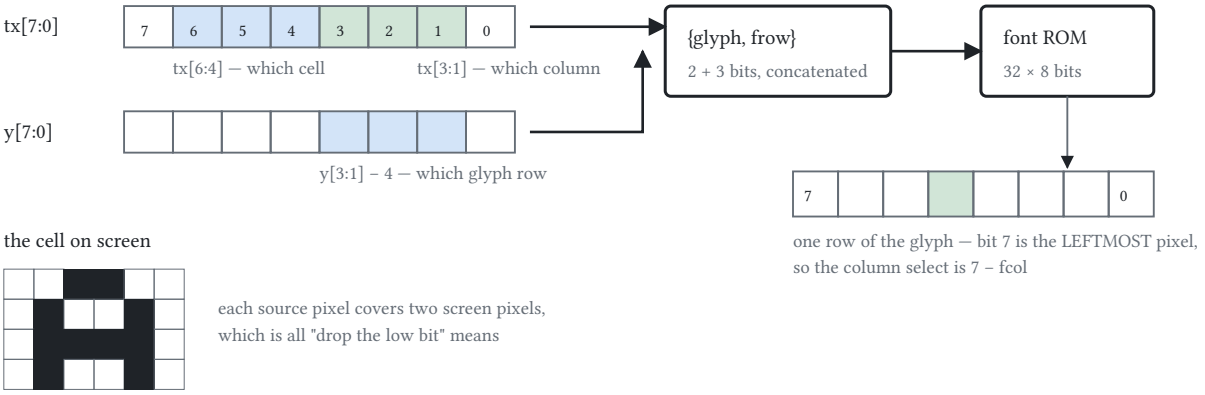
Lesson 06 established that you can drive a display without storing a frame, as long as you can *compute* the pixel at any coordinate the moment it is needed. Text is the first interesting thing to compute.

A bitmap font is a tiny ROM: each character is eight bytes, each byte a row, each bit a pixel. So "is this pixel part of a letter?" becomes a chain of arithmetic on the current raster coordinate — which character cell is x inside, which glyph is in that cell, which row does y select, which bit does x select.

That chain is exactly what a hardware text-mode display controller does, and what a GPU's texture unit does in more dimensions: a coordinate in, an address and a bit-select out.

Theory: make the arithmetic disappear

```
wire [2:0] ci    = tx[6:4];        // which character cell: divide by 16
wire [2:0] frow  = y[3:1] - 3'd4;  // which glyph row: divide by 2
wire [2:0] fcol  = tx[3:1];        // which column: divide by 2
wire font_bit = font[{glyph, frow}][3'd7 - fcol];
```



Every step is a bit slice, a concatenation or a subtraction of a constant: no divider, no multiplier, no logic worth the name.

A 16-pixel cell and a 2× scale are chosen so the arithmetic disappears. Ten pixels and 3× would be identical in behaviour, and expensive.

A raster coordinate becoming a ROM address and a bit select. Every arrow is a bit slice or a concatenation, which is why none of it costs anything.

Every one of those is **free**, and that is the point:

- `tx[6:4]` is a divide by 16 — a wire rename, no logic at all. Choosing a 16-pixel cell width (a power of two) is what makes it free; 10 pixels would have cost a divider.
- `tx[3:1]` is the same trick for the 2× scale: dropping the low bit makes every source pixel cover two screen pixels.
- `font[{glyph, frow}]` concatenates a 2-bit glyph index and a 3-bit row into a 5-bit address. Address arithmetic by *concatenation* rather than multiplication, because the glyph stride is 8.
- `3'd7 - fcol` reverses the bit order, because bit 7 of the font byte is the *leftmost* pixel while x increases rightward.

"Choose your constants so the arithmetic disappears" is one of the highest-leverage habits in hardware design. The same design with a 10-pixel cell and a 3× scale is functionally identical and dramatically more expensive. Lesson 08 meets the same idea with a price attached, when a framebuffer's `y × 96 + x` turns out to cost two hardware multipliers.

Watch out. `frow = y[3:1] - 3'd4` quietly relies on the text band starting at $y = 24$. Change the band's position and this constant must change with it — and

the two facts are forty lines apart with nothing connecting them. A constant derived from another constant should be *derived*, not duplicated.

What it cost, and the question it raises

MEASURED `text_top` — the whole design
`make synth`

resource	count
LUT4	185
TRELLIS_FF	72
CCU2C	22
PFUMX	50
L6MUX21	25

185 of the 85F's 83,640 LUT4 — 0.22% of the part.

Set that beside lesson 06's display controller — 108 LUT4, 72 flip-flops, 22 carry cells, 29 muxes — and the interesting row is the second. **No new flip-flops at all.** The text added no state, only combinational arithmetic. The extra 77 LUT4 are the coordinate chain and the font ROM.

And there is the puzzle this lesson ends on: **a 32-byte ROM built out of logic, not memory.** The part has 208 block RAMs sitting idle and the synthesiser used none of them.

The reason is size. One block RAM holds 18,432 bits; spending a whole one on 256 bits would waste 98.6% of it. The decision flips somewhere between 32 bytes and a few kilobytes — and knowing roughly where is a useful instinct, which is what lesson 08 is for.

On what the test does not check

The testbench verifies that the SPI stream is well-formed and that the initialisation sequence is correct. It does *not* check that the letters are the right shape — that would mean reconstructing a 96 × 64 image and comparing against a reference.

This gap is deliberate and worth sitting with. **A passing test tells you only what it was written to ask.** The course returns to this repeatedly: lesson 09 finds a testbench asserting something false, lesson 11 finds a mutation that survives for a good reason, lesson 12 finds a hole in its own vectors, and lesson 13 finds a bug that 161 passing checks had missed.

Lesson 08 — Memories

The idea

The part has 208 `DP16KD` blocks — 18,432 bits each, about 3.7 Mbit in total, true dual-port, with a **registered output**. You cannot instantiate one in this course's style; you write an array and the synthesiser infers it, under three conditions:

- the array is a `reg [W-1:0] mem [0:DEPTH-1]`
- reads and writes happen inside `always @(posedge clk)`
- **the read result goes into a register**

That last one is not a style preference. A block RAM's output is registered *inside the block*, physically. A combinational read is something a `DP16KD` cannot do, so asking for one takes block RAM off the table — permanently, at any size.

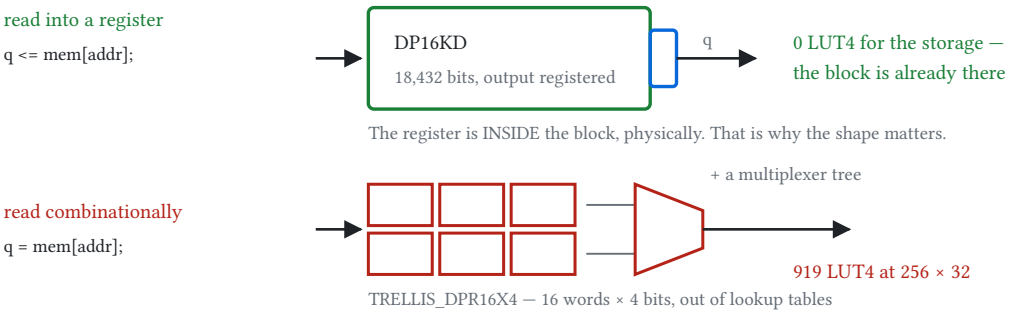
What it cost

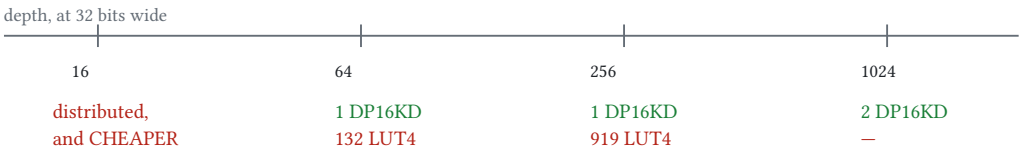
MEASURED `mem_top` — the whole design
`make synth`

resource	count
LUT4	80
TRELLIS_FF	61
CCU2C	27
PFUMX	6
DP16KD	2

80 of the 85F's 83,640 LUT4 — 0.1% of the part.

Two block RAMs out of the part's 208, and the rest of the design is the test harness around them. The row to notice is the one that is *absent*: almost no LUT4 go into the storage itself, because the storage is not built from logic at all.





Synchronous once it is big enough to want a block. A shape that cannot become one removes the decision for good.

One line of Verilog chooses the primitive — and at 256 entries the two lines are 919 LUT4 apart.

The measurement

MEASURED the same array, two ways, at W = 32

depth	synchronous read	combinational read
16	8 `TRELLIS_DPR16X4` + 32 `TRELLIS_FF`	**8 `TRELLIS_DPR16X4`**
64	**1 `DP16KD`**	32 `TRELLIS_DPR16X4` + 132 LUT4 + 96 muxes
256	**1 `DP16KD`**	128 `TRELLIS_DPR16X4` + **919 LUT4** + 672 muxes
1024	**2 `DP16KD`**	—

Three readings, and the third is the one people miss.

At 256 entries, one line of difference costs 919 LUTs. The combinational version becomes *distributed* RAM — 16 words of 4 bits built from a logic cell's lookup tables — plus an enormous multiplexer tree.

At 1024 × 32 the synchronous version is two block RAMs and nothing else. 32,768 bits, and a `DP16KD` holds 18,432. Clean inference, zero LUTs.

And at 16 entries both give distributed RAM, with the combinational one *cheaper* — it needs no output register. Small asynchronous arrays have a legitimate home, and that is exactly how a 16-entry register file gets built.

So the rule is not "always synchronous". It is **synchronous once it is big enough to want a block, and know where that line is.** Lesson 11 depends on this: RV32I's 32-entry register file needs a combinational read for a single-cycle CPU, and that turns out to be the right choice rather than a compromise.

Lesson 07′s puzzle, answered

MEASURED `font_small` — 32 bytes, lesson 07′s font

```
make synth M=font_small
```

resource	count
LUT4	9
TRELLIS_FF	5
PFUMX	4

MEASURED `font_full` — 1,024 bytes of identical code

```
make synth M=font_full
```

resource	count
DP16KD	1

Same Verilog, different size, opposite answer.

Lesson 07′s font became logic because it was *too small to be worth a block*. The right conclusion is not "use logic for small memories" — it is: **write the memory in the shape that can become a block RAM and let the synthesiser decide on size.** Writing a shape that cannot removes the decision for good.

`$readmemh` runs at elaboration time

```
initial $readmemh("font.hex", mem);
```

The synthesiser reads the file *during synthesis* and bakes the contents into the bitstream. The hex file is therefore a **source file** — change it and you must resynthesise. This is the path lesson 13 uses to load a program, and lesson 15 uses for a program compiled by GCC.

Watch out. A ROM whose contents are all zero is not a ROM — it is a constant, and the synthesiser folds it away entirely. The lesson's font file begins with ASCII 0–31, which have no glyphs, so synthesising it at 256

entries emits *no memory at all*. "My ROM disappeared" usually means the file was not found, was empty, or was genuinely all one value across the range used.

Two traps found while building it

A bug this course hit. Registered write data. The memory self-test registered `din` inside the clocked block, so by the time the RAM performed the write on the following edge, `addr` had already advanced. *Every location held its neighbour's value* and the read-back reported 128 errors out of 128.

A memory's address, write-enable and write-data must be presented in the **same cycle**. Register some and not others and they arrive skewed — and the memory faithfully stores the skew.

A bug this course hit. A constant multiply that was not free. The framebuffer's address is `y * 96 + x`. A comment in the first draft predicted the synthesiser would reduce a constant multiply to shifts. It did not — it allocated **two `MULT18X18D` hardware multipliers**. Writing `(y << 6) + (y << 5)` explicitly, since 96 = 64 + 32, removes them entirely.

`wy * 96 + wx`	1 DP16KD + **2 MULT18X18D** + 8 CCU2C
`(wy<<6) + (wy<<5) + wx`	1 DP16KD + 14 CCU2C + 24 LUT4

If you know the cheap form, write the cheap form — then check.

Watch out. Uninitialised memory reads as **X** in simulation. The ECP5 happens to zero its block RAM from the bitstream, but Verilog does not promise it, so a test that relies on initialisation is testing the simulator. The lesson's framebuffer test writes its zeros explicitly, after an earlier version failed for exactly this reason.

Why a framebuffer wants two ports

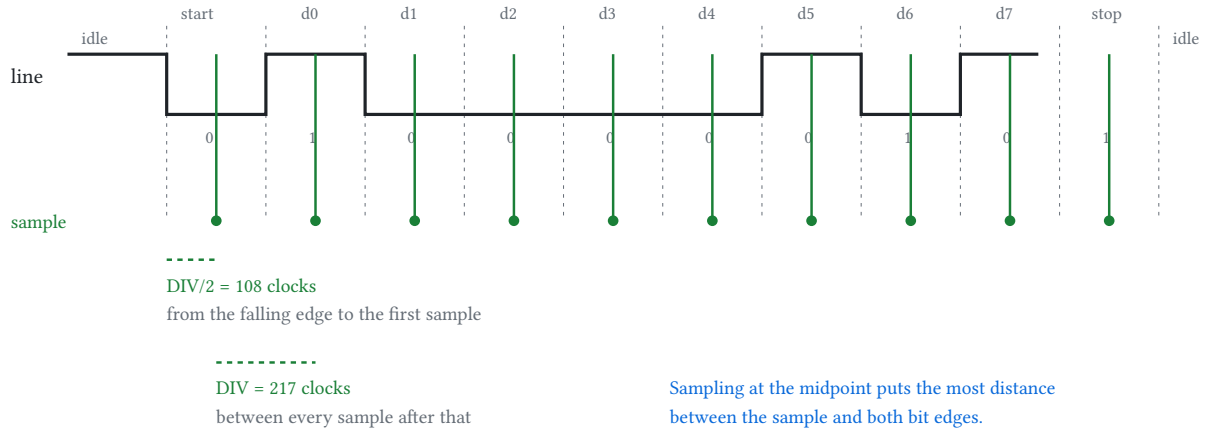
A display memory is written one way and read another, simultaneously: written by coordinate in whatever order the drawing code likes, and read in raster order relentlessly, because the display never stops. One port cannot serve both without one side waiting, which is largely why the `DP16KD` exists at all.

Lesson 09 — UART

The idea

There is **no clock on the wire**. Two chips, two crystals, one signal line each way. Everything depends on both ends agreeing in advance how long a bit lasts, and on one edge to say when to start counting.

That is the whole of 8N1: a start bit (the line falls from idle-high), eight data bits least significant first, and a stop bit. Ten bit periods, no handshake, no acknowledgement.



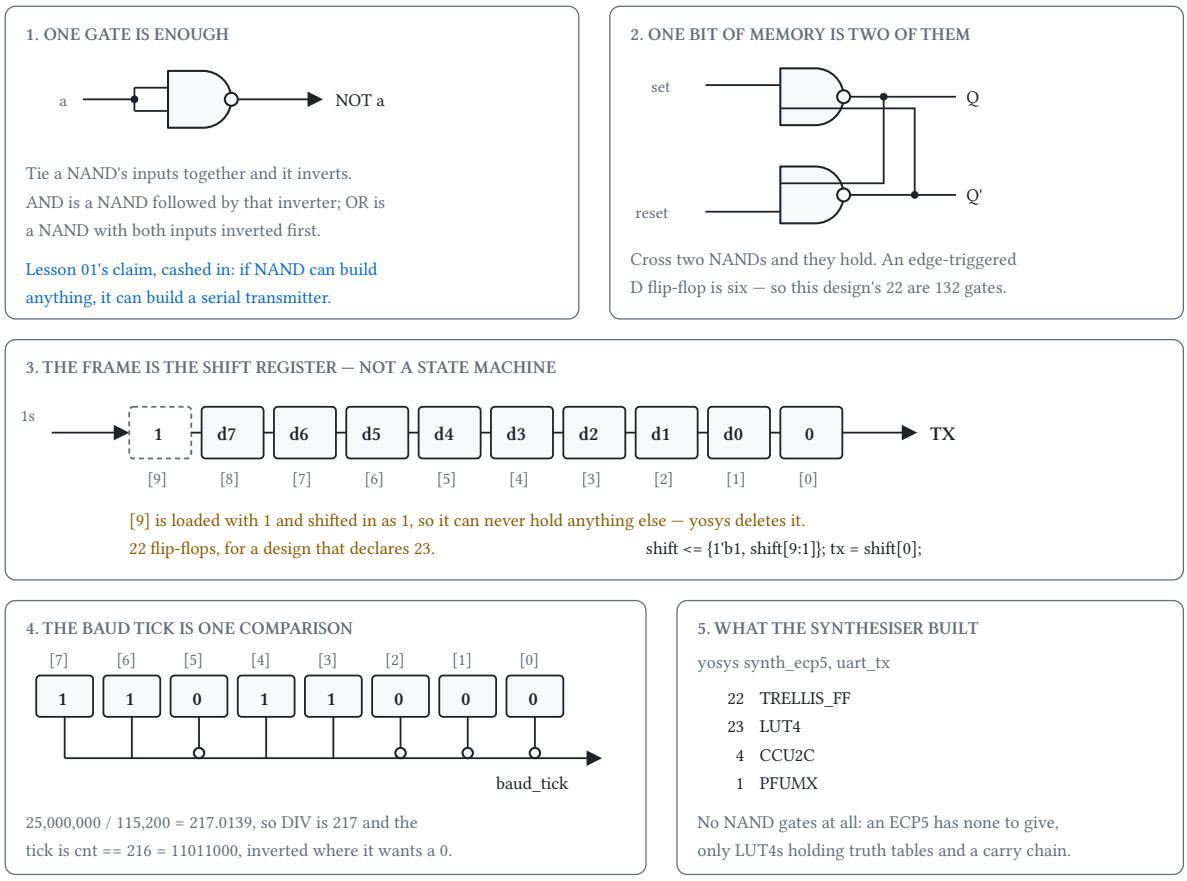
An 8N1 frame, with the receiver sampling at the midpoint of every bit.

Baud rate is a counter modulus

$$25,000,\frac{000}{115},200 = 217.0139$$

So the divisor is 217, giving 115,207 baud — **0.0064% fast**. Over the 9.5 bit periods to the middle of the stop bit that accumulates to 0.06% of a bit, which is nothing. Even 216 or 218 would be fine, at 4.5% of a bit.

The same transmitter, taken down to the gate



`uart_tx.v` at 25 MHz and 115,200 baud, taken down to the gate. The frame is the shift register, the baud tick is one comparison against 216, and the cell counts are measured rather than idealised.

Lesson 01 claimed that NAND alone can build any function. This is that claim cashed in on something that does a job: tie a NAND's inputs together and it inverts, cross two of them and they hold a bit, and six make an edge-triggered flip-flop — so the 22 flip-flops below are 132 gates before a single bit of logic.

Two details in the figure are worth more than the gates. `shift[9]` is loaded with 1 and has 1 shifted into it, so it can never hold anything else and yosys deletes it: a design that declares 23 flip-flops synthesises to 22. And the part is innocent of NAND gates entirely — an ECP5 has none to give, only LUT4s holding truth tables and a carry chain in dedicated silicon. The gate-level picture is how you reason; it is not what gets built.

Where the textbook had to be overruled

Every UART reference says: build a clock at 16 times the baud rate and sample on its eighth tick. That is how a 16550 works, because a 16550 is *given* a 16× baud clock. Here it does not work.

MEASURED 16× oversampling is unavailable on a 25 MHz crystal
115,200 × 16 = 1,843,200 Hz, and 25,000,000 / 1,843,200 = 13.5634 — not an integer.

divisor	baud	error	drift by the stop bit
13	120,192	+4.33%	**+41% of a bit**
14	111,607	−3.12%	**−30% of a bit**

Either way the stop bit gets sampled inside bit 7 and the frame is lost.

The 16× clock is a good idea *given a crystal chosen for it*, and a bad idea given this one. So: count the bit period directly — which divides cleanly — and sample at the half-way point. Same underlying idea, which is to put the sample as far from both edges as possible, implemented to fit the clock you actually have.

The receiver re-checks the start bit at its own midpoint, which is what makes a glitch harmless: a brief disturbance looks like a start bit for a few clocks, and confirming it at the midpoint rejects it rather than reading eight bits of noise.

Watch out. The receive line comes straight from a pin, driven by *another chip's oscillator*. It is asynchronous, so it goes through a two-flop synchroniser before anything looks at it. Lesson 04 exists for this moment, and this is the first lesson that genuinely needs it.

The honest limit

A receiver must absorb some rate error, because two crystals never agree. Driving frames at a range of rates against a correctly-configured receiver:

MEASURED tolerance, and what lies beyond it

sender error	result
+6.4% … −4.8%	accepted, byte correct
beyond that	**accepted, byte WRONG, no framing error**

The out-of-tolerance frames are *not rejected*. Far enough off, the stop-bit sample lands somewhere inside the data, finds a 1 by chance, and the frame passes its only check.

A framing error is not a reliable corruption detector. It catches many cases; it does not catch all of them. Detecting the rest is a *protocol* job — a parity bit, or a checksum over a packet. Hardware that cannot distinguish "correct" from "plausible" needs software that can.

A bug this course hit. The lesson's testbench originally asserted that mistimed frames were never silently corrupted — and passed. The sweep above disproves that claim. **A test that asserts something false and passes is worse than no test**, so it now asserts what is true and the tolerance window is documented with its numbers instead.

What a loopback test does not prove

Running all 256 byte values through transmitter into receiver is worth having — it finds bit-order and framing mistakes. But both ends share a clock and a divisor, so it **cannot find a timing problem**, which is the entire difficulty of a UART. The tolerance sweep is the test that does.

A bug this course hit. A second testbench bug, worth knowing because the pattern recurs. The bit-period measurement asserted `start` and *then* set up a wait for the line's falling edge. The edge had already happened, so it measured the next one — the five-zero run inside the letter `A` — and reported 40 clocks for an 8-clock bit. The transmitter was correct; the test was looking in the wrong place. Measuring edges in a testbench needs `fork`/`join` so the monitor is already waiting before the stimulus starts.

What it cost

MEASURED `uart_top` — both halves, the echo and the LEDs

`make synth`

resource	count
LUT4	91
TRELLIS_FF	71
CCU2C	8
PFUMX	16
L6MUX21	7

91 of the 85F's 83,640 LUT4 — 0.11% of the part.

MEASURED `uart_tx` — the transmitter

`make synth M=uart_tx`

resource	count
LUT4	23
TRELLIS_FF	22
CCU2C	4
PFUMX	1

MEASURED `uart_rx` — the receiver

`make synth M=uart_rx`

resource	count
LUT4	48
TRELLIS_FF	33
CCU2C	4
PFUMX	9
L6MUX21	4

The receiver costs more than twice the transmitter. That is the resource footprint of not knowing when the bits start: the transmitter only has to count, while the receiver has to work out *when to look*.

Watch out. The pin names are from the **FTDI's** point of view. `ftdi_rxd` is where the FTDI *receives*, so the FPGA drives it; `ftdi_txd` is where the FTDI transmits, so the FPGA reads it. `output ftdi_rxd` and `input ftdi_txd` are correct and look backwards. This is the classic place to lose an afternoon.

PART IV

The processor

The ALU, the register file, decode and immediates, then the single-cycle core. Ends with a program of your own running on the board, and one instruction followed through it end to end.

Lesson 10 — The ALU	Where the shifter turns out to be 80% of it, and exhaustive testing dies
Lesson 11 — The register file	Two read ports, one write port, and `x0`
Lesson 12 — Decode and control	Immediates that look like vandalism, and why they are not
Lesson 13 — A single-cycle RV32I processor	The core, running assembled programs — plus one instruction, end to end

Lesson 10 — The ALU

The idea

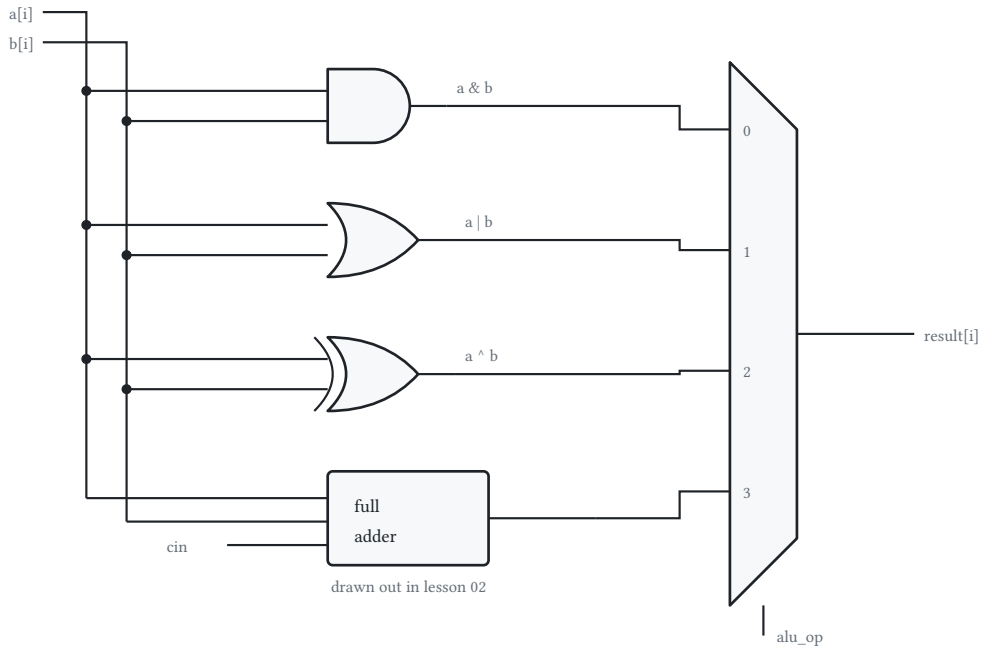
From here the modules are not illustrations. They are the processor's parts.

RV32I's integer ALU has **ten operations and not one more**: add, sub, three shifts, two comparisons, and three bitwise operations. A CISC machine might have hundreds. Everything else is built by the compiler out of these — negation is `sub` from `x0`, bitwise NOT is `xor` with −1, branch-if-greater is branch-if-less with the operands swapped.

That austerity is the "reduced" in RISC, and it is a claim about what *hardware* should cost rather than about what programmers should write.

The encoding is a gift

The operation selector is `{instruction[30], funct3}` — four bits. Only **two** operations need bit 30, and they are exactly the two pairs where one is the signed or arithmetic variant of the other: `ADD`/`SUB` and `SRL`/`SRA`. So lesson 12's decoder pulls this field straight out of the instruction word with no translation table at all.



This slice, thirty-two times over, is the cheap half of the ALU: every operation is computed on every cycle and the opcode decides which one leaves. Each bit needs only its own two operand bits.

The shifter is the expensive half precisely because it cannot be drawn this way: result[i] depends on a bit of a that the shift amount chooses.

One bit of the ALU in gates. Every operation is computed on every cycle; the opcode only decides which result is allowed out.

Theory

One adder does four jobs

`a - b` == `a + (~b) + 1` in two's complement, so subtraction is the *same* adder with `b` inverted and the carry-in set. And both comparisons fall out of that same subtraction for free:

Unsigned less-than is the borrow. Carry-out is 1 when there was no borrow, i.e. when `a` ⩾ `b`. So `a < b` is simply `~carry` — no comparator at all.

Signed less-than is where ALUs go wrong. It is not a comparison of the raw bits, and it is not simply the sign of the difference either, because the difference can overflow:

```
wire overflow = (a[W-1] ^ b[W-1]) & (sum[W-1] ^ a[W-1]);
wire lt       = sum[W-1] ^ overflow;
```

Watch out. The shortcut that nearly works: "if the signs differ, the answer is just `a`'s sign; a negative number is always less than a positive one." That is true, and implementing it that way passes almost every test you would write by hand. It fails exactly where the subtraction overflows — `INT_MIN` against `INT_MAX`, and `0` against `INT_MIN`.

The shift trap

```
wire [W-1:0] sra = $signed(a) >>> sh;
```

`>>>` is an arithmetic shift **only if its left operand is signed**. Writing `a >>> sh` on an unsigned wire silently performs a *logical* shift. It compiles, simulates, lints clean, and is wrong for every negative input. `$signed(a)` is the whole fix, and this is the most common ALU bug there is.

Exhaustive testing dies here

A 32-bit ALU has $2^{32} \times 2^{32} \times 10$ input combinations — about 4.6×10^{19}. At a billion a second, 1,460 years.

What replaces it is the pattern for everything from here on:

1. **Directed edge cases.** Twelve interesting operands — zero, one, −1, `INT_MAX`, `INT_MIN`, `INT_MIN+1`, −2, alternating patterns, shift amounts 31 and 32 — crossed against each other for all ten operations. 1,440 vectors covering every extreme against every other extreme.
2. **Randomised comparison against a reference model.** 20,000 vectors. The model uses Verilog's operators and `$signed`; the design builds the same answers from an adder and explicit overflow logic. Two different routes to one answer.
3. **An independent second implementation, where a model would prove nothing.** This is the subtle one. The design's shifts *are* Verilog's shift operators, so checking them against the same operators tests nothing whatsoever. They are cross-checked instead against an iterative shifter that moves one place per clock — a genuinely different implementation that can disagree.

Mutation testing

A test that has never caught a bug is unproven. So: break the design deliberately and check the test notices.

MEASURED three mutations, three catches, with the exact operands named

`sra = a >> sh` — drop `$signed`	caught: `a=ffffffff b=1 → 7fffffff`
`lt = sum[W-1]` — drop overflow	caught: `a=0 b=80000000 → 1`
`ltu = carry` — wrong polarity	caught: `a=0 b=1 → 0`

Mutation testing is the only way to know a testbench has teeth. Lesson 09 found a testbench asserting something false and passing; this is the systematic version of that check, and every testbench from here on is worth mutating once.

What it cost

The surprise is in the proportions, so each piece is reported on its own.

MEASURED `alu` — the whole thing

`make synth M=alu`

resource	count
LUT4	885
CCU2C	17
PFUMX	281
L6MUX21	127

885 of the 85F's 83,640 LUT4 — 1.06% of the part.

MEASURED `adder` — 32-bit addition

```
make synth M=adder
```

resource	count
CCU2C	17

Essentially free: no LUT4 at all.

MEASURED `shift_barrel` — single-cycle shifting

```
make synth M=shift_barrel
```

resource	count
LUT4	667
PFUMX	262
L6MUX21	139

About 80% of the ALU, for one operation out of ten.

MEASURED `shift_iterative` — the same shifts, 31 cycles

```
make synth M=shift_iterative
```

resource	count
LUT4	76
TRELLIS_FF	40
CCU2C	3
PFUMX	32
L6MUX21	1

The adder — what you would assume an ALU is mostly about — is seventeen carry cells, because the fabric has dedicated carry hardware. Shifting has *none*, so a barrel shifter is a tree of multiplexers five layers deep and 32 wide, three times over for the three shift kinds.

The iterative shifter is **seven times smaller** and takes up to 31 cycles. On an 8-bit microcontroller that is the usual choice. In a single-cycle CPU it is not available at all, so lesson 13 pays for the barrel shifter whether it likes it or not.

"Which operations are expensive" is a property of the fabric, not intuition. Addition is free here and shifting is not; on a chip with no carry chain the answer would invert.

What it cost in time — the ceiling lesson 13 will hit

A registered 32-bit ALU closes at **115.94 MHz**, with the critical path running through the adder's carry chain *and* the ten-way result multiplexer. Both on one path — which is precisely the argument lesson 16 makes for splitting them across pipeline stages.

Lesson 11 — The register file

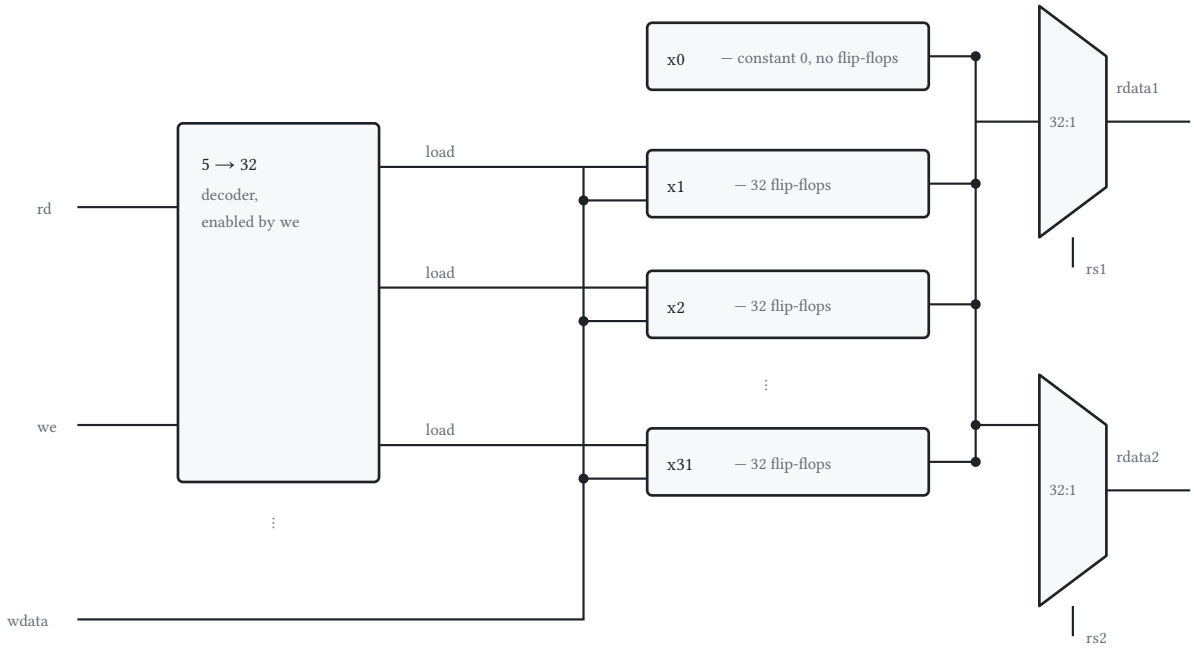
The shape is dictated by the instruction set

```
add rd, rs1, rs2
```

One instruction, **two source operands read in the same cycle**, one result written. So: two read ports and one write port. If the file had three read ports, instructions could have three sources. If it had one, R-type instructions as we know them could not exist.

The instruction set and the register file are the same design decision seen from two sides. Register-file ports are expensive, and that cost is visible in the shape of every RISC instruction set — this is the clearest example in the course of hardware constraints writing software syntax.

Two reads and one write, every cycle, and no clock anywhere on the read path.
The write side is a decoder; the read side is two 32-to-1 multiplexers, and that is where the logic goes.



The register file as gates and flip-flops: a decoder on the write side, two large multiplexers on the read side, and `x0` wired to zero.

`x0`

Reads of `x0` return zero; writes are discarded. Two lines of Verilog, and they simplify the whole instruction set. None of the following are real instructions — they are assembler conveniences that exist *because* one register reads as zero and swallows writes:

what you write	what it assembles to
`mv rd, rs`	`addi rd, rs, 0`
`li rd, imm`	`addi rd, x0, imm`
`nop`	`addi x0, x0, 0`
`not rd, rs`	`xori rd, rs, -1`
`neg rd, rs`	`sub rd, x0, rs`
`beqz rs, L`	`beq rs, x0, L`
`j L`	`jal x0, L` — throw the return address away
`ret`	`jalr x0, 0(x1)`

One register sacrificed; eight instructions avoided. `jal x0, L` is the neatest: a jump-and-link whose link register is the bit bucket *is* an unconditional jump, so the ISA needs no `j` opcode at all.

Combinational reads, and why lesson 08 agrees

A single-cycle CPU must read its operands, compute, and write back inside one clock period, so the read has to be combinational. Lesson 08 showed that rules out block RAM — which sounded like a warning, and here is the correct answer, because lesson 08's *other* finding applies: at a few tens of entries the fabric builds arrays from lookup tables as distributed RAM, cheaply and with no latency.

What it cost

MEASURED `regfile` — combinational read, as RV32I needs

`make synth M=regfile`

resource	count
LUT4	266
PFUMX	132
L6MUX21	66
TRELLIS_DPR16X4	32

266 of the 85F's 83,640 LUT4 — 0.32% of the part.

MEASURED `regfile_bram` — the same file, asking for block RAM

`make synth M=regfile_bram`

resource	count
LUT4	214
TRELLIS_FF	108
PFUMX	6
L6MUX21	4
TRELLIS_DPR16X4	32

No `DP16KD` in either. Thirty-two entries is too small for the synthesiser to spend a block on, so the "block RAM version" got distributed RAM too — the same storage, plus 108 flip-flops of pipeline registers, plus a cycle of latency, for nothing.

MEASURED at 1024 registers the trade flips decisively

combinational read	**3,660 LUT4** + 454 muxes + 1,024 `TRELLIS_DPR16X4`
registered read	**4 `DP16KD`** + 192 LUT4 + 131 flip-flops

Same two modules, same code, opposite conclusion — decided entirely by size. A vector machine's register file lives on the right of that table; RV32I's lives firmly on the left.

Read-during-write returns the old value

During cycle *N* an instruction reads its operands, computes, and its result is written by the clock edge at the *end* of cycle *N*. So a read of the register being written returns the **old** value — which is not a limitation but a requirement: an instruction must see its operands as they were before it ran.

Watch out. This confuses people in both directions. Some expect a write-to-read bypass and are surprised by the old value; others add one and silently change what their CPU computes. The testbench asserts the old value explicitly, so adding a bypass later is a visible, deliberate change.

Bypass becomes *necessary* when a read and a write belong to different instructions in flight at once — lesson 16's pipeline, and the block-RAM variant above, where latency forces the issue.

A mutation that should survive

Four mutations; three caught. The fourth — *allowing writes to `x0`* — survives, and that is the interesting result.

It survives because `x0`'s behaviour is enforced in **two independent places** and either one alone suffices. With the write guard gone, `regs[0]` holds garbage that the read mask makes unobservable. Remove *both* and the test fails immediately.

So the correct reading is not "the test is weak" but "the code is deliberately redundant" — and no behavioural test can distinguish them, because there is no behavioural difference. A test reaching inside to inspect `regs[0]` would be testing the implementation rather than the contract.

A surviving mutation means either your test missed something, or your code has belt-and-braces redundancy. Check which before fixing anything. Lesson 12 meets the other kind.

Exhaustive testing is briefly back

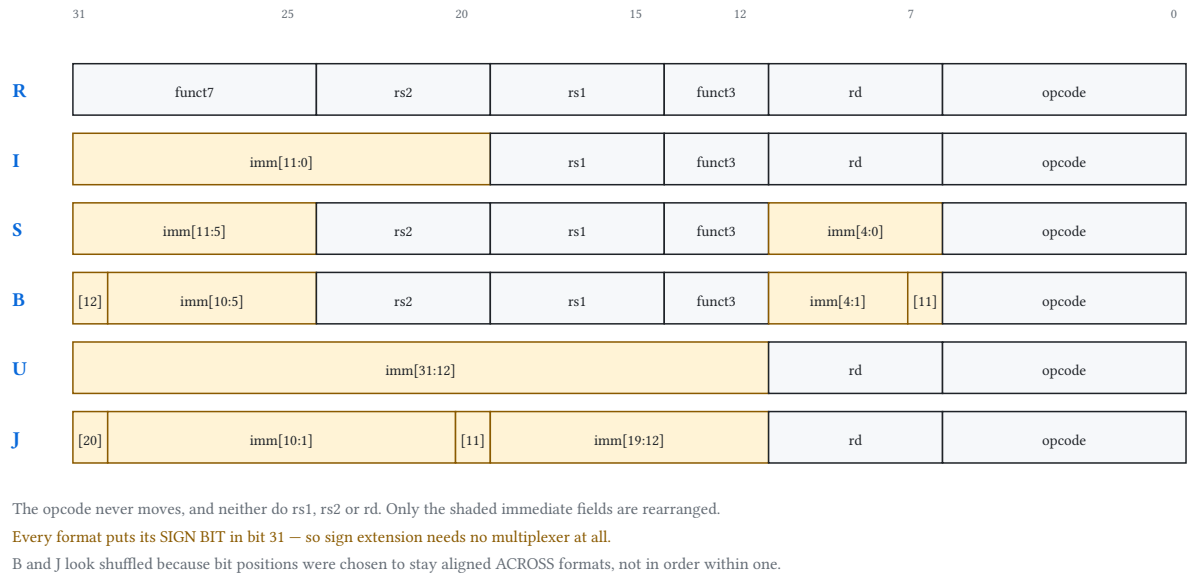
A register file has 32 addresses, not 2^{32} values, so the thing that matters — *which register* — can be swept completely. All 961 pairs of distinct registers are read simultaneously, which is what catches a second port quietly sharing the first one's address decoder.

Lesson 12 — Decode and control

Fixed fields, decoded in parallel

Every other field's *meaning* depends on the opcode, so the opcode must be decodable before anything else — which is why it is always `inst[6:0]`. Putting it at a fixed position means the register fields, `funct3` and the immediate can all be extracted **in parallel** with decoding the opcode rather than after it.

The register fields never move either: `rs1` is always `inst[19:15]`, `rs2` always `inst[24:20]`, `rd` always `inst[11:7]`. So lesson 13 addresses the register file *before the instruction has been understood at all.*



The opcode never moves, and neither do rs1, rs2 or rd. Only the shaded immediate fields are rearranged.

Every format puts its SIGN BIT in bit 31 — so sign extension needs no multiplexer at all.

B and J look shuffled because bit positions were chosen to stay aligned ACROSS formats, not in order within one.

The six formats. Only the shaded immediate fields are rearranged; every other field stays where it is.

The immediates look like vandalism

B puts `imm[11]` in `inst[7]` while `imm[12]` is in `inst[31]`. J shuffles its bits out of order entirely. Read as a table it looks arbitrary.

Two rules explain all of it:

1. **The sign bit is always `inst[31]`.** Every format. So sign extension needs no multiplexer at all — just replicate one fixed wire.
2. **Each immediate bit comes from as few different places as possible.** `imm[10:5]` is `inst[30:25]` in I, S, B *and* J. `imm[4:1]` is `inst[24:21]` in I and J, `inst[11:8]` in S and B.

Rule 2 is *why* B and J look shuffled: bit positions were assigned to stay aligned **across** formats, not in order **within** one.

MEASURED count the multiplexer inputs and the design appears
Of the 32 immediate output bits, counting how many distinct instruction-word positions feed each — a bit with *N* sources needs an *N*-input multiplexer:

sources	bits
1 — no multiplexer at all	**24**
2	7
3	1

41 multiplexer inputs across all 32 bits, against an upper bound of 160 if the five layouts were uncorrelated — 26% of it.

And the single bit needing three sources is `imm[11]` — precisely the one that looks most arbitrary in the specification. The designers put the ugliness in *one place* rather than spreading it everywhere.

"The encoding is weird" and "the encoding is carefully optimised" turn out to be the same observation from two sides. You cannot tell which from reading the specification; you can tell immediately from the multiplexer count.

Note also that B and J immediates are implicitly multiplied by two — bit 0 is always zero and is not stored, because instructions are 2-byte aligned. One extra bit of branch range, free.

The trap

Lesson 10's ALU takes `{inst[30], funct3}`, which falls straight out of the instruction word for R-type. For I-type it very nearly does:

```
addi x1, x2, -1
```

has `imm = 0xFFF`, so `inst[31:20]` is all ones and **`inst[30]` is 1**. Pass `{inst[30], funct3} = {1, 000}` to the ALU and it performs **SUB** instead of ADD — so `addi` with any negative immediate silently subtracts.

`inst[30]` is only meaningful for `OP-IMM` when `funct3 == 101`, distinguishing `SRLI` from `SRAI`. Everywhere else in `OP-IMM` it is part of the immediate and must be forced to zero.

Watch out. This is invisible in an R-type-only test, and any immediate at or beyond 1024 — or any negative one — trips it. Without the fix, lesson 13's first program computes nonsense.

A bug this course hit. A bug in the first draft of this decoder, caught by the linter rather than by a test: `alu_op` was assigned from `{alu_bit30, funct3}`, and a *separate* wire applied the "loads and stores must add" override — and was never connected to anything. Every load would have asked the ALU for whatever `funct3` happened to mean. One assignment per output.

Testing a decoder

The vectors are **real encodings**, generated by a small assembler and each labelled with the assembly it came from — every format, the sign-extended extremes of each, and eight malformed instructions that must raise `illegal`.

The legal contrast cases matter as much as the illegal ones. A decoder that rejected everything would pass all the illegal tests, so encodings sitting right next to them — `blt`, `bge`, `sra` — are asserted *not* to be flagged.

Watch out. One of the lesson's own illegal vectors was simply wrong. `0x00314863` was written intending "branch with `funct3=011`"; it actually has `funct3=100`, which is `blt` and perfectly legal. The decoder correctly declined to flag it and the test failed. Hand-computing instruction encodings is error-prone — compute them with a script and print the fields back to check.

A bug this course hit. Seven mutations; six caught immediately. The survivor forced S-type `imm[0]` to zero — and was invisible because **every store offset in the vectors happened to be even**. Adding `sb x3,7(x2)` catches it at once.

Unlike lesson 11's surviving mutation, which was benign because the design was redundant, this one was a *real missing test*. Both outcomes are useful, and telling them apart is the skill.

What it cost

MEASURED `decode_top` — the whole design

`make synth`

resource	count
LUT4	53
TRELLIS_FF	14
PFUMX	14
L6MUX21	9

53 of the 85F's 83,640 LUT4 — 0.06% of the part.

MEASURED `immgen` — the immediate generator

`make synth M=immgen`

resource	count
LUT4	70
PFUMX	17
L6MUX21	10

MEASURED `decode` — the control decoder

`make synth M=decode`

resource	count
LUT4	45
PFUMX	8
L6MUX21	3

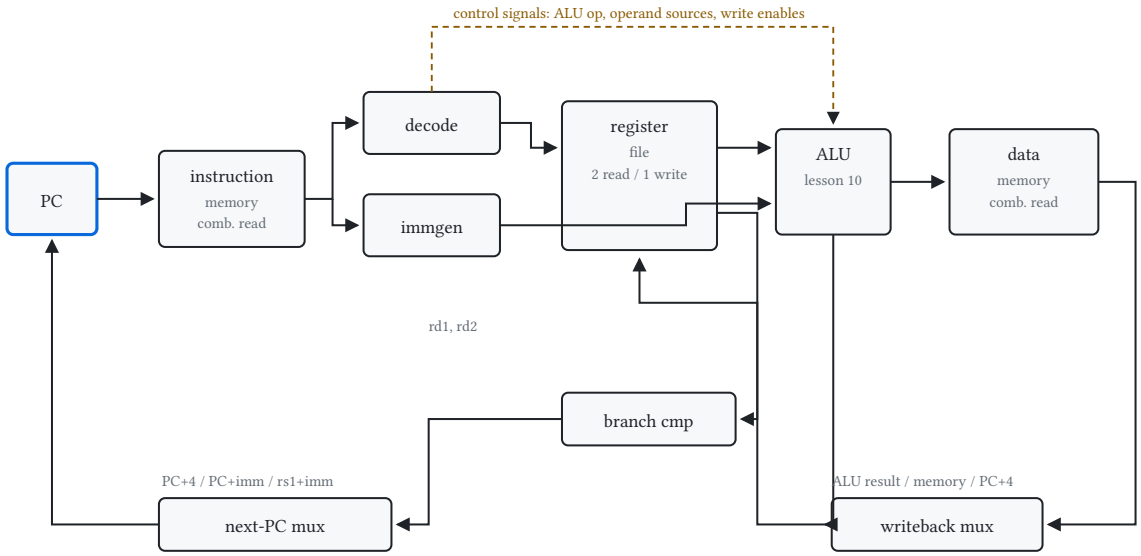
The immediate generator costs more than the entire control decoder — consistent with the multiplexer counts. Both are tiny beside lesson 10's ALU at 885 LUT4, and both vanish inside the board demo above, which only has to show a few of their outputs on LEDs.

Lesson 13 — A single-cycle RV32I processor

Nothing here is new

The ALU is lesson 10. The register file is lesson 11. The decoder and immediate generator are lesson 12. The only new logic is a branch comparator, the load/store aligners, and the next-PC multiplexer.

A CPU is not a new kind of thing. It is a state machine (lesson 05) whose state is a program counter, wired to parts you have already built and tested separately. That is the most demystifying idea in the course, and it is why those four lessons came first.



The datapath. Everything shown happens within one clock period.

Why single-cycle first

There are no pipeline registers between the stages, so every wire goes where the diagram says it goes. Read the source top to bottom and it is the picture, in order. Lesson 16 pipelines this exact core, and the comparison is the point of doing it this way round.

Harvard, and not as a preference

There are **two** memories, instruction and data, both read combinationally. That is not a style choice: a single-cycle CPU must fetch an instruction *and* access data in the same cycle, and one memory with one port cannot do both.

Real machines present one address space and manage with a single memory by pipelining — fetch in one stage, memory access in a later one — or by putting separate caches over shared memory. Here the simplest thing that works is two memories, and saying so is better than pretending it was taste.

The new parts

A separate branch comparator

Branches need six comparisons. The ALU could do them, but in a single-cycle CPU **the ALU is busy computing the branch target in the same cycle**. So a separate comparator, which is cheap and keeps the ALU's output free.

Note what RV32I omits: there is no `BGT` or `BLE`. `bgt a, b, L` assembles as `blt b, a, L` — the *assembler* swaps the operands and the hardware needs no extra cases. Six conditions, not ten.

Byte and halfword access

Memory is 32 bits wide; the ISA has `LB`, `LH`, `LW`, `LBU`, `LHU`, `SB`, `SH`, `SW`. So a byte store must change *one* byte and leave the other three alone — which is why the memory has per-byte write enables — and a byte load must select and extend. `LB` sign-extends and `LBU` zero-extends; both exist because C's `char` may be either and a compiler must be able to ask for exactly one.

The bug only running software could find

The CPU ran on the first attempt — 109 cycles, reaching `halt` — and got most answers right. Every failure was an **X**, and all of them traced to `LUI`.

A bug this course hit. Lesson 12's decoder carried a comment saying *"rs1 is x0 by convention for LUI"*, and the datapath relied on it. **That is false.** `LUI` has no `rs1` field at all: `inst[19:15]` is part of its 20-bit immediate. So the ALU was adding the immediate to whatever register those five bits happened to name — an undefined register — and the X propagated into every value built with a large constant.

Lesson 12's own testbench checked that `LUI` asks the ALU for ADD, and it does. **It never checked where the operands came from.** 161 passing decoder checks and a mutation-tested testbench did not find this; eighty-eight instructions of real software found it in one go.

Unit tests verify the parts. Integration finds the assumptions between the parts. Both are necessary and neither substitutes for the other. The decoder now has an explicit "operand A is zero" output, and its testbench asserts it for `LUI` and asserts it is *not* set for `add` or `auipc`.

Testing through the architectural interface

The test program stores every result to a known memory address, and the testbench reads data memory rather than peeking at the register file.

That tests the CPU through the interface *software actually uses*, so a core that computes correctly but *stores* wrongly still fails — which is also all that software could tell you. The program ends by setting a flag, and writes a marker if any branch goes the wrong way; the test requires the flag and forbids the marker.

What it cost

MEASURED `cpu` — the processor itself

```
make synth M=cpu
```

resource	count
LUT4	1,765
TRELLIS_FF	32
CCU2C	80
PFUMX	547
L6MUX21	226
TRELLIS_DPR16X4	32

1,765 of the 85F's 83,640 LUT4 — 2.11% of the part.

Thirty-two flip-flops in an entire CPU, and they are the program counter. All the state a single-cycle machine has is its PC: the register file lives in distributed RAM and everything else is combinational. Lesson 16 will add hundreds of flip-flops, and that is precisely what pipelining *is*.

MEASURED `cpu_top` — the core plus its memories and board wiring

`make synth`

resource	count
LUT4	1,268
TRELLIS_FF	40
CCU2C	52
PFUMX	316
L6MUX21	153
TRELLIS_DPR16X4	42

Fewer LUT4 than the core it contains, which is not a misprint: synthesised on its own, `cpu` must build every output it exposes, while inside `cpu_top` only the parts that drive something survive. **A module's cost depends on what is asked of it** — so a per-module number only means anything alongside the command that produced it, which is why every table in this book carries one.

What it cost in time

MEASURED the number this lesson exists to produce

32-bit ALU alone (lesson 10)	115.94 MHz
the whole single-cycle core	**61.24 MHz**

Adding fetch, register read, memory access and writeback to the ALU's path **roughly halves the clock**. Every instruction — even `nop` — pays the full critical path, because every instruction takes exactly one cycle and that cycle must be long enough for the slowest one.

That is the entire argument for pipelining, now with a number attached. Lesson 16 splits this path into five stages so each is short, and pays for it in hazard logic.

Exactly what this core implements

The core executes the **RV32I base integer instruction set, less the privileged and system parts**. Being precise about that matters: "RISC-V" on its own claims rather a lot, and lesson 14 is about not claiming what you have not shown.

implemented	All nine base opcodes and every instruction under them: `LUI`, `AUIPC`, `JAL`, `JALR`, the six branches, the five loads, the three stores, the nine register-immediate operations and the ten register-register operations — the full RV32I arithmetic, logic, shift, compare, branch, jump and load/store set, with byte and halfword access and sign or zero extension
not implemented	`FENCE` (there is nothing to order), `ECALL` and `EBREAK`, and the entire CSR and privileged architecture — so no traps, no interrupts, no machine mode. Those are the planned lesson 17
on an illegal instruction	the decoder raises `illegal`, the register file and data memory are held off, and the program counter stops advancing. The core halts rather than trapping, because trapping needs CSRs it does not have
misaligned access	**not detected.** A misaligned load or store does something well-defined by the aligner and wrong by the specification. A real core would trap; this one has nowhere to trap to
memory	Harvard: separate instruction and data memories, both block RAM on the FPGA, no caches and no bus. One cycle, always

So: RV32I user-level integer code, assembled by this repository's assembler, runs correctly — and that is the claim lesson 14 then takes seriously for the ALU. It is not a compliance claim about the whole core, and the official riscv-arch-test suite has not been run against it.

On the board

Set the switches to 2 and you are reading the result of `addi t2, t0, -1`, which must be **9**. Had lesson 12's trap gone unfixed it would read 11.

This is a RISC-V processor you built, running a program you wrote, on a chip you configured. Worth a moment.

One instruction, end to end

Lessons 10 to 13 built an ALU, a register file, a decoder and a processor to hold them. This section builds nothing. It takes a *single* instruction and walks it through the machine you now have, naming every file it touches on the way — and then answers the four questions that always follow.

```
add x5, x6, x7
```

Add the contents of `x6` and `x7`, put the result in `x5`. With `x6 = 10` and `x7 = 20`, afterwards `x5 = 30` and the two sources are untouched. That is the whole of what the instruction set promises; everything below is how the promise is kept.

If the previous four lessons worked, nothing here will be new — and that is the point of reading it. A processor is not a thing you understand all at once; it is four things you already understand, arriving in order.

Thirty-two bits that mean "add"

The instruction is not text and is never parsed. It is one 32-bit number, and the hardware reads fields out of it by position. R-type, from bit 31 down:

field	bits	width	for this instruction
`funct7`	31–25	7	`0000000`
`rs2`	24–20	5	`00111` — x7
`rs1`	19–15	5	`00110` — x6
`funct3`	14–12	3	`000`
`rd`	11–7	5	`00101` — x5
`opcode`	6–0	7	`0110011` — register-register integer op

Assembled by this repository's own assembler, which is where the number below comes from rather than from arithmetic done by hand:

```
$ python3 tools/rvasm.py add.s --list
00000008  007302b3   add  t0, t1, t2
```

MEASURED the same thirty-two bits, three ways

```
0000000  00111  00110  000  00101  0110011
funct7    rs2    rs1  funct3  rd    opcode

0000 0000 0111 0011 0000 0010 1011 0011   =   0x007302B3
```

`t0`, `t1` and `t2` *are* `x5`, `x6` and `x7` — the ABI names the assembler accepts are the same five bits.

Why the fields are where they are

Three questions that the layout above always provokes, and they have better answers than "because the specification says so".

Why is the opcode at the bottom?

Because every other field's *meaning* depends on it, so it must be readable first — and a field at a fixed position is readable immediately rather than after something else has been worked out. Lesson 12 made the consequence concrete: the register fields never move either, so lesson 13's core addresses the register file *before the instruction has been understood at all.* Decode and register read happen at the same time, not one after the other.

Why five bits for a register number?
Because $2^5 = 32$, and RV32I has 32 registers. Five bits span `00000` (x0) through `11111` (x31) and there is nothing left over.

Watch out. Address width and data width are different things, and conflating them is a common first mistake. Each register is 32 bits *wide*, and its address is 5 bits *long*. The file holds 32 × 32 = 1,024 bits of state and needs 15 address bits across its three ports to reach them — lesson 11 measured the whole thing at 266 LUT4 and 32 distributed-RAM cells.

Why seven bits for `funct7`?
Not because anything needs seven. **Because seven is what is left.**

```
32 - 7 (opcode) - 5 (rd) - 3 (funct3) - 5 (rs1) - 5 (rs2)  =  7 bits
```

The other five fields are sized by what they must hold, they are nailed to fixed positions so they can be read in parallel, and `funct7` is the remainder of a 32-bit word after the useful fields have been placed. The architecture then spends that space distinguishing a handful of operation pairs — `ADD` from `SUB`, `SRL` from `SRA` — and leaves the rest for later.

MEASURED how much of funct7 this course's hardware actually uses: one bit

```
assign alu_op = needs_add ? 4'b0000 : {alu_bit30, funct3};
```

From `12-datapath/decode.v`. The ALU operation is chosen by `funct3` plus **`inst[30]` alone** — one bit of the seven. The other six are never consulted to decide *what to do* — they are only checked to decide whether the instruction is *legal* at all, which is what that lesson's `bad_funct7` test does.

So: the encoding spends seven bits, the hardware reads one, and the gap between those two numbers is encoding space held in reserve. That is a design choice about the future, not a requirement of arithmetic.

The journey

1. Fetch
The program counter — 32 flip-flops, and in a single-cycle core the only state the processor has — addresses instruction memory. `imem_addr = pc`, the memory answers combinationally, and `inst` is the 32-bit number above.

2. Decode, and register read, together
`decode.v` splits the instruction into control signals; `immgen.v` extracts an immediate that this instruction will not use. Simultaneously, because the register fields are at fixed positions, `rs1 = inst[19:15]` and `rs2 = inst[24:20]` go straight to the register file. No instruction has been "understood" yet and the operands are already being fetched.

The decoder does not add. It identifies, and it asserts `reg_we`.

3. Execute
The ALU of lesson 10 receives `a = 10`, `b = 20` and `alu_op = {0, 000}`, and produces 30. Underneath, a 32-bit adder — which lesson 02 built out of gates at 212 cells and lesson 10 got for 17 `CCU2C` and *no LUT4 at all* by writing `+` and letting the fabric's carry chain do it.

4. Write-back
`wb_src` selects the ALU result over the memory and PC+4 paths, `rd = x5`, and at the clock edge the register file takes the write. Had `rd` been `x0`, the register file would have discarded it — which is lesson 11's one-line mechanism and the reason `jal x0, label` needs no special case anywhere.

5. The next instruction
Not a branch, not a jump: `pc ⩽ pc + 4`. The same clock edge that writes `x5` advances the program counter, which is what makes this a single-cycle machine.

What it looks like in the simulator
Running those five steps on the lesson 13 core, printing the core's own signals at each negative edge:

MEASURED observed, on the core from lesson 13

```
pc=00000004  inst=01400393  rs1=x0(0)   rs2=x20(x)  alu_y=20  rd=x7  reg_we=1
pc=00000008  inst=007302b3  rs1=x6(10)  rs2=x7(20)  alu_y=30  rd=x5  reg_we=1
pc=0000000c  inst=10502023  rs1=x0(0)   rs2=x5(30)  alu_y=256 rd=x0  reg_we=0
pc=00000010  inst=0000006f  ...
x5 stored to 0x100 = 30
```

The middle line is the instruction. `inst` is the number this section opened with, `rs1` and `rs2` carry 10 and 20, the ALU produces 30, `rd` is x5 and `reg_we` is high. The line below it is the store that proves it: `rs2=x5(30)` — the register really does hold 30 on the next cycle.

Watch out. Look at the third line's `reg_we=0` and the first line's `rs2=x20(x)`. The store writes no register, so its `rd` field is not a destination; and `li` has no second source, so the bits sitting in `rs2` name a register nobody asked for and the value read is *X*. Both are harmless — nothing downstream looks — and both are the hardware reading fields it cannot yet know are meaningless. That is the price of decoding in parallel, and it is cheap.

The control signals, and what they are called here

Textbooks name these after the classic MIPS datapath. This course names them after what they select, and they are the same signals:

textbook	this course	for `add x5, x6, x7`
`RegWrite`	`reg_we`	1 — the result is kept
`ALUSrc`	`alu_b_imm`	0 — operand B is `rs2`, not the immediate
`ALUOp`	`alu_op`	`0000` — add
`MemRead`	`mem_read`	0
`MemWrite`	`mem_write`	0
`MemToReg`	`wb_src`	`WB_ALU` — the ALU result, not memory
`Branch`	`branch`	0 — the next PC is `pc + 4`

The ten operations that share this opcode

`0110011` is not "add". It is "register-register integer operation", and `funct3` with one bit of `funct7` picks which:

instruction	`funct7`	`funct3`	
`ADD`	`0000000`	`000`	the one above
`SUB`	`0100000`	`000`	same `funct3`; `inst[30]` is the difference
`SLL`	`0000000`	`001`	
`SLT`	`0000000`	`010`	signed
`SLTU`	`0000000`	`011`	unsigned
`XOR`	`0000000`	`100`	
`SRL`	`0000000`	`101`	logical
`SRA`	`0100000`	`101`	arithmetic; `inst[30]` again
`OR`	`0000000`	`110`	
`AND`	`0000000`	`111`	

Ten instructions, one opcode, and exactly two places where `funct7` matters. Note how little of the ALU's cost this table represents: lesson 10 measured the whole ALU at 885 LUT4, of which the barrel shifter — `SLL`, `SRL`, `SRA`, three rows — is about 80%.

How many instructions does RISC-V have?

There is no single answer, and the reason is the architecture's main idea.

RV32I	40 instructions in the unprivileged base: `LUI`, `AUIPC`, `JAL`, `JALR`, six branches, five loads, three stores, nine register-immediate and ten register-register operations, plus `FENCE`, `ECALL` and `EBREAK`
M	multiply and divide — lesson 17 adds these
Zicsr	the CSR instructions — lesson 17 adds these too
A, F, D	atomics, single- and double-precision floating point
C	16-bit compressed encodings
B, V	bit manipulation, vectors

A RISC-V processor implements the base plus whichever extensions it chooses, and says so in the `misa` register. **This course's core implements RV32I less `FENCE`, `ECALL` and `EBREAK` at lesson 13**, adds `Zicsr` and `M` at lesson 17, and is honest in both lessons about what it has not been tested against.

Modularity is why `MUL` hides under opcode `0110011` with `funct7 = 0000001` rather than taking an opcode of its own. Opcodes are the scarce resource in a fixed 32-bit encoding; an extension that needed one would be an extension that could not coexist with another. Lesson 12's `funct7` legality check is exactly what makes `MUL` illegal on a core without M — which is what the specification requires, and why that check was worth writing before there was anything to reject.

This is one instruction, on one implementation, explained completely — not a specification. Instruction legality, privileged behaviour and extension encodings are defined by the official RISC-V ISA manuals, and anything built to interoperate should be checked against those. What it answers is "but what actually happens?", and the answer is four modules you built, in order, inside one clock period.

PART V

Verification and extension

A formal proof of the ALU, a computer that runs compiled C, the same core pipelined and measured against the first, then CSRs, traps and a timer interrupt.

Lesson 14 — Formal verification	Proving the ALU rather than sampling it — and a proof that was worthless
Lesson 15 — A system on chip	A memory map, peripherals, and a program compiled by GCC
Lesson 16 — Pipelining	Five stages: what the clock buys, and what CPI costs
Lesson 17 — CSRs, traps and the M extension	A machine that reports an illegal instruction instead of halting

Lesson 14 — Formal verification

Why this lesson exists here

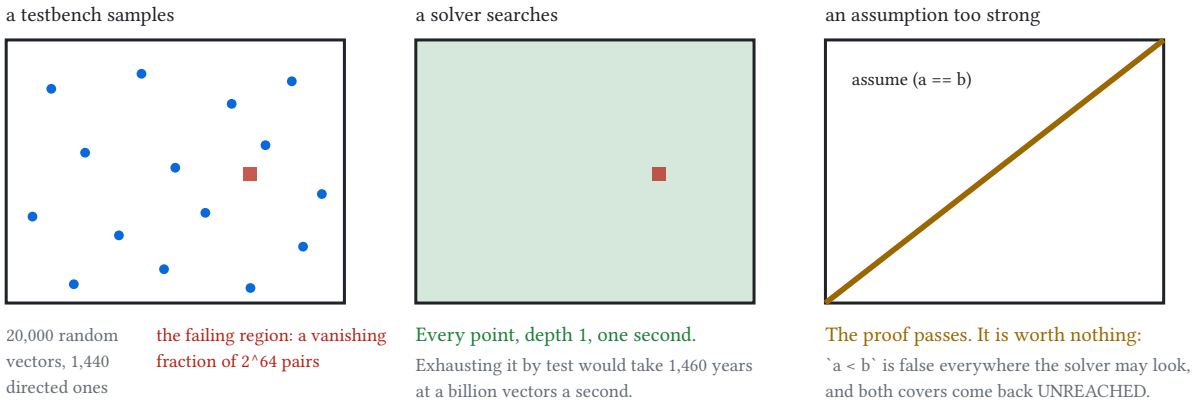
Lesson 13 ended with twenty-four passing checks and a CPU that ran. It also ended by finding a bug that **161 passing decoder checks and a mutation-tested testbench had missed** — because the bug was in an assumption no test had asked about.

That is the argument for formal methods in one sentence. A testbench tells you a design works for the inputs you imagined. For a processor that is nowhere near good enough: the bugs live in combinations nobody imagines, and the space is far too large to sample.

The idea

A bounded model checker takes a property — "this signal is never high while that one is low" — and either proves it holds for every reachable state within k cycles of reset, or hands you a concrete trace where it fails.

The word **bounded** is doing real work. A proof to depth 20 says nothing about a bug that needs 21 cycles to manifest. Depth is traded against solver time, and choosing it is part of the craft.



What formal does that testing cannot is search rather than sample — and nothing in a passing result tells you the assumptions were right.

The same input space three times: sampled, searched, and shrunk to a line by an assumption that was slightly too strong.

Three kinds of statement:

`assert`	something that must always be true. The thing being proved.
`assume`	a constraint on the inputs. **The dangerous one**: assume too much and you prove something about a machine nobody will build.
`cover`	a request for a trace reaching some state. The antidote to the above — if a `cover` is unreachable, your assumptions have excluded reality.

`riscv-formal`

An instruction-level specification of RV32I that an implementation can be checked against, by Claire Wolf. The core exposes a trace interface reporting what it retired each cycle — which instruction, which registers it read and wrote, what values — and the framework checks each retirement against the specification.

What it proves, instruction by instruction: that the result matches the ISA, that the program counter advanced correctly, that no register was disturbed that should not have been, and that memory access was properly aligned.

This is also where you find out which of the earlier lessons were quietly wrong. A formal proof over a CPU exercises the ALU, the register file and the decoder far harder than their own testbenches did — and it does so without anyone having thought of the case.

What it actually did

MEASURED the ALU, proved rather than sampled

Lesson 10 computed that checking all $2^{32} \times 2^{32} \times 10$ ALU inputs would take **1,460 years** at a billion a second, and settled for 1,440 directed vectors plus 20,000 random ones.

The proof covers every one of them in **one second**. The ALU is combinational, so depth 1 examines the entire input space — for combinational logic, "bounded" costs nothing at all.

The register file, which has state, is proved for **every sequence of twelve operations** over every address and every data value, in 58 seconds.

The register file proof uses the technique worth taking away:

```
(* anyconst *) reg [4:0] watched;   // the SOLVER picks the address
always @(posedge clk)
    if (we && rd == watched && watched != 0) shadow <= wdata;
always @(*)
    if (shadow_valid && rs1 == watched) assert (rd1 == shadow);
```

`anyconst` makes this a proof rather than a spot check: the solver chooses `watched` and must satisfy the property for *every* choice, so one shadow register establishes a fact about all thirty-two.

Catching a real bug

The lesson ships a deliberately broken ALU — lesson 10's, with signed less-than using the shortcut that omits the overflow term. That failing region is a vanishing fraction of 2^{64} pairs, and 20,000 random vectors would very likely miss it.

MEASURED the solver found it in under a second

```
a = 0x80008000   signed  -2147450880
b = 0x57FE8001   signed   1476296705
signed:  -2147450880 < 1476296705  →  1
broken ALU answered:  0
```

Signs differ, the difference overflows, and the shortcut is wrong — precisely the case lesson 10 described. Nobody told the solver to look there.

Watch out. The first thing a formal tool finds is usually a bug in your *property*. The first run of this proof failed with the design perfectly correct: the property used `y[sh-1:0]`, which is not a legal variable part-select, against a nonsensical right-hand side. Expect this.

The failure mode that makes formal dangerous

A bug this course hit. A proof can pass and be worth nothing. The lesson's `vacuous.v` uses the **real** ALU, makes **true** assertions, and **passes** — because of one line:

```
assume (a == b);
```

With the operands forced equal, `a < b` is false for every input the solver may consider. The assertion becomes `0 == 0`, the overflow logic is never exercised, and an ALU returning constant zero for SLT would pass identically.

Nobody writes that deliberately. They write an assumption to exclude an input the design is not meant to handle, it is slightly too strong, and the hole is invisible — because a passing proof looks the same either way.

The antidote is `cover`, and it is not optional. Running the vacuous example's cover task reports both covers **unreached**, which is the only signal that the proof above was hollow.

A formal proof is only as good as its assumptions, and nothing in a passing result tells you whether they were right. The real ALU proof carries seven covers for this reason — that signed and unsigned comparison genuinely disagree, that `SRA` differs from `SRL`, that addition wraps — and all seven are reached.

What it cost

Nothing, and the absence is deliberate: this lesson has no design of its own. It proves lesson 10′s ALU and lesson 11′s register file, both of which report their own area. What this lesson spends is **solver time**, and `run-proofs.sh` prints it.

What this lesson does not do

`riscv-formal` itself — checking a whole core against an instruction-level specification of RV32I — needs the core to expose a retirement trace interface and the riscv-formal repository alongside. That is a lesson's work on its own and is not here yet. What *is* here is the technique it rests on, applied to the two components the CPU is mostly built from.

Formal verification is not a replacement for testing. It is bounded, and writing a property that is both true and useful is a skill. What it does that testing cannot is **search** rather than sample.

Lesson 15 — A system on chip

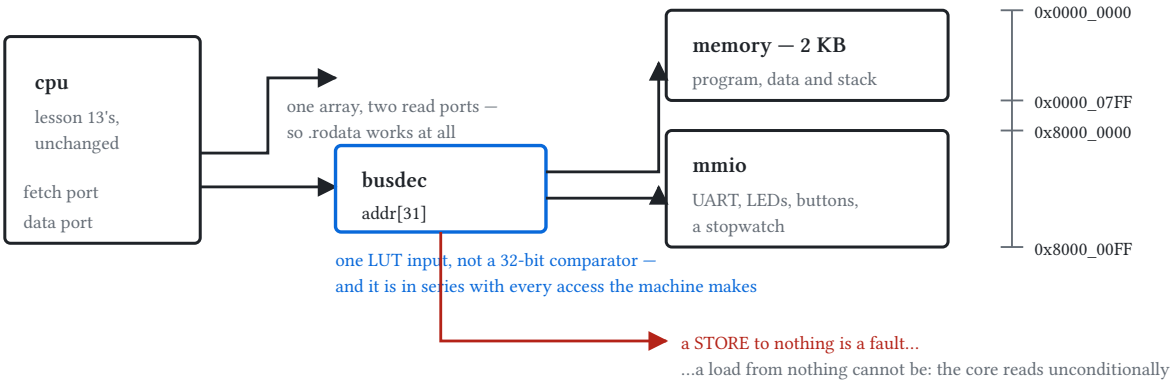
Where a CPU becomes a computer

Lesson 13's memories ignored the high address bits entirely, so `0x0000_0100` and `0xDEAD_0100` hit the same word. The memory *aliased* across the whole 32-bit space — which means every address was valid, which means no mistake could ever be reported.

A real system **decodes** the high bits to decide which device an address belongs to. That decoding is the memory map, and it is the difference between a CPU and a computer.

address	device	
`0x0000_0000–0x0000_07FF`	memory	2 KB: program, data and stack
`0x8000_0000–0x8000_00FF`	I/O	the peripheral registers
everything else	*nothing*	and an access there is reported

The I/O window sits at the top bit on purpose. `addr[31]` alone separates I/O from memory, so a select line costs **one LUT input** rather than a 32-bit comparator. Real memory maps are laid out with exactly this in mind: the reason a peripheral block sits at `0x4000_0000` or `0xE000_0000` and never at `0x1234_5678` is that the decoder is in series with every memory access the machine will ever make.



No `ready` anywhere: a single-cycle core cannot wait, so every target must answer combinationally — which is why this memory is 2,756 LUT4 and not one block RAM. The missing handshake is lesson 13's decision, arriving as a constraint.

The whole machine: lesson 13's core, one array with two read ports, a decoder that is one address bit wide, and a fault for everything unmapped.

A peripheral is then just an address that does something when written. There is no I/O instruction in RV32I — no `IN`, no `OUT` — so the UART of lesson 09 becomes two addresses, reached with the same `LW` and `SW` as everything else.

This is the lesson where the hardware/software boundary stops being something you read about and becomes something *you chose*. The address decoder and the C header are the same decision written twice, once for each side, and nothing but your own care makes them agree.

One memory, two ports

Lesson 13 had two memories because a single-cycle CPU fetches an instruction *and* accesses data in the same cycle, and one memory with one port cannot do both. The fix is not a second memory. It is **a second port**.

One array, one address space, two read ports — fetch, and data. Software now sees a flat memory in which a string constant can be read by the instruction executing a few words away from it, which is what C expects and what makes `.rodata` work at all.

Watch out. One address space means a store can change an instruction. On this machine self-modifying code even works, because the fetch is asynchronous and there is no cache and no pipeline holding a stale copy. On

any real machine it does not, and the instruction-cache flush that makes it work is one of the reasons a JIT compiler is harder than it looks.

The bus has no `ready`, and that absence is the design

A real bus — AXI, Wishbone, even the simplest handshake — lets the *target* say "not yet". That is what makes a slow peripheral, a cache miss or an off-chip memory possible.

This bus cannot. The lesson 13 core has no way to wait, because a single-cycle CPU finishes the instruction in the cycle it started. **Every target here must answer combinationally**, which is why the memory is built from LUTs instead of block RAM, and why nothing slower than a register can ever be attached.

So the missing `ready` is not a simplification for teaching. It is the single-cycle decision of lesson 13, arriving two lessons later as a constraint on what can be plugged in.

A read may not have a side effect

On almost every real UART, reading the data register consumes the byte. That design is impossible here, and the reason is worth more than the convenience it costs.

A register with a side effect on **read** has to know that a read really happened. The lesson 13 core reads its data memory *unconditionally* — every instruction puts its ALU result on the address bus whether or not it is a load, and the writeback multiplexer decides whether the returned word is used. So if reading the receive register consumed a byte, any instruction whose arithmetic happened to produce `0x8000_0008` would eat a character — and `li t0, -2147483640` is exactly that number.

Consuming the byte is therefore a **write**, to a separate acknowledge register, because a write strobe is only ever asserted by a real store.

Watch out. A bus whose master cannot say *"this is a real read"* may not have read side effects anywhere in its address map.

The same asymmetry decides what can be reported. A store to an unmapped address is unambiguously a mistake, and the decoder raises a fault. A *load* from one cannot be, so it quietly reads whatever the aliased word holds. Lesson 16's core brings its memory-read signal out — a pipeline has to know which stage is accessing memory anyway — and lesson 17 gives the fault somewhere to be reported *to*.

The toolchain path

```
riscv64-elf-gcc  →  objcopy  →  .hex  →  $readmemh  →  the memory array
```

Lesson 08 established that `$readmemh` runs at elaboration time, so the compiled program is baked into the bitstream. Changing the C means recompiling *and* resynthesising.

`volatile` tells the compiler two things it otherwise assumes are false: that this location can change without the program changing it, and that writing to it has an effect beyond storing a value. Drop it from a status-register poll and gcc is entirely within its rights to emit one load and an infinite loop — because nothing in the program changes that variable, so why read it twice? That is not a compiler bug. It is the C abstract machine being told the truth and acting on it, and `volatile` is how you say *this address is hardware*.

Before `main`

With no operating system, something must set up the stack pointer and zero the `.bss` section before `main` runs. That is `crt0`, it is about twenty instructions here, and you can read all of them.

What is *missing* from it is the interesting part. On a microcontroller booting from flash, `.data` has two addresses — it lives in flash and runs in RAM — and `crt0` must copy it across. That copy loop is in every embedded `crt0` you will ever read, and it is absent here, because `$readmemh` placed the initialised data into the memory array from the bitstream before the clock ticked once. Load address and run address are the same address.

That is a real property of a soft core rather than a shortcut, and it says exactly what would change to boot this machine from the board's SPI flash: the copy loop comes back, and so does a linker script with `AT>` in it.

What it cost

MEASURED `soc_top` — the whole computer

`make synth`

resource	count
LUT4	4,821
TRELLIS_FF	146
CCU2C	114
PFUMX	1,460
L6MUX21	593
TRELLIS_DPR16X4	544

4,821 of the 85F's 83,640 LUT4 — 5.76% of the part.

Not one block RAM. The 85F has 208 of them, each holding 18,432 bits; this design needs 16,384 bits of memory — which would fit in a single one — and uses none, because a single-cycle core needs its memory to answer in the same cycle and lesson 08 proved that rules a `DP16KD` out.

MEASURED `memory` — 2 KB, two asynchronous read ports

`make synth M=memory`

resource	count
LUT4	2,756
PFUMX	800
L6MUX21	465
TRELLIS_DPR16X4	512

More than half the computer, to hold 2 KB. Lesson 08's table predicted it: a 256-entry asynchronous memory measured 919 LUT4 against a single `DP16KD` for the synchronous version, and this is the same arithmetic at twice the size with two read ports.

MEASURED `busdec` — the entire memory map

`make synth M=busdec`

resource	count
LUT4	20
CCU2C	10
PFUMX	7
L6MUX21	4

Twenty LUT4. The thing that turns a CPU into a computer is the cheapest module in the design by two orders of magnitude — which is what putting the I/O window at `addr[31]` buys.

MEASURED `mmio` — UART, LEDs, buttons, stopwatch

`make synth M=mmio`

resource	count
LUT4	200
TRELLIS_FF	113
CCU2C	24
PFUMX	66
L6MUX21	33

113 flip-flops, against 32 in the entire CPU. A peripheral is almost all state; a single-cycle datapath is almost all combinational logic.

What it cost in time, and the size this lesson had to ship

This is where the lesson stopped being a write-up and became a measurement, because the first version of it **did not work on the board**.

The memory is read asynchronously, so every fetch and every load drags a multiplexer tree the full depth of the array, in series with the whole single-cycle datapath. The deeper the memory, the slower the machine.

MEASURED memory size is now a timing parameter

memory	LUT4	max clock, three placer seeds	at 25 MHz
4 KB — 1024 words	7,263	25.14 / 24.44 / 24.40 MHz	**fails 2 runs in 3**
2 KB — 512 words	**4,821**	**27.28 / 28.21 / 28.00 MHz**	**passes**
1 KB — 256 words	4,007	31.73 / 30.63 / 30.49 MHz	passes

```
make synth M=soc_top P="MEM_WORDS 1024"
```

4 KB of memory does not fit in 25 MHz on this machine. Not "is tight" — does not fit. The board's oscillator is 25 MHz and `nextpnr` reports `FAIL`, which means the design may do anything at all on hardware, including working until it warms up.

Three things in that table matter more than the numbers.

Three seeds, not one. `nextpnr`'s placer is randomised, and the 4 KB build passes on seed 1 and fails on seeds 2 and 3. A design that meets timing by 0.5% has not met timing; it has got away with it once. Quoting a single placement as *the* frequency is the most common way to publish a number nobody else can reproduce.

Doubling the memory cost 10% of the clock. Memory size has not been a timing parameter in any earlier lesson. Lesson 08's warning was about area; this is the same decision returning as speed.

The linker script is where this lands on the programmer. `link.ld` says `LENGTH = 2K` because of the row above, and a program that outgrows it does not get a slower machine, it gets a link error. That is the hardware telling the software what it may be.

MEASURED the cost of becoming a computer

32-bit ALU alone (lesson 10)	115.94 MHz
single-cycle core, small Harvard memories (lesson 13)	61.24 MHz
the same core, as a computer	**28 MHz**

Building a computer around the core halved its clock again. Lesson 13 paid for putting the whole datapath into one cycle; this lesson pays for putting a real memory and an address decoder on both ends of it. Nothing was done badly — this is what the single-cycle decision costs once the machine is big enough to run C.

That is the argument for pipelining, stated in megahertz and in LUT4. Lesson 16 takes the same core, cuts it into five stages, and gets both numbers back: the memory becomes two block RAMs, and the clock nearly doubles.

On the board

MEASURED what the hardware actually sent

```
RV32I SoC
deadbeef
bss ok
K
CYC 0000025e
DONE
```

Seven consecutive runs, every one byte-identical to the string the testbench expected at the time — and `CYC 0000025e` is the same 606 cycles the simulation produced. The delay loop takes exactly as long on real silicon as it does in Icarus.

The program has since gained a longer greeting, the `x+y` and opcode lines, and a carriage return on every line; the capture above is left as it was taken, because a measurement is a record of what happened and not a thing to edit afterwards. The cycle count is the part that was being measured, and it has not moved.

The capture also contained two greetings cut off after a character or two, which is worth more than it looks. At 115,200 baud a character takes 87 microseconds, so a greeting that managed one `R` before restarting means two resets about that far apart — from one press of the button.

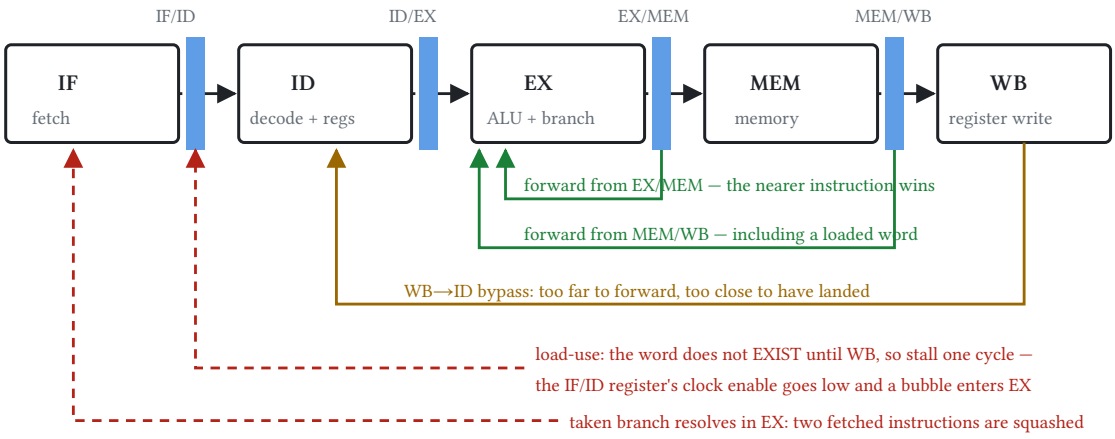
That is contact bounce, and it is the thing lesson 03 built a debouncer for. The board demo wires the reset button straight to `rst` with nothing in between, because a repeated reset is harmless — which is exactly the judgement lesson 03 asks you to make rather than debouncing everything by reflex. The evidence that it was the right call, and the evidence that the bouncing is real, are the same six lines of serial output.

What this machine does not do

faults	a store, and only a store, to an unmapped address is latched and shown. A load from one reads an aliased word and nobody is told
traps	none. An illegal instruction halts the core, as in lesson 13, because trapping needs CSRs that lesson 17 adds
the bus	one master, no arbitration, no `ready`, no burst. Everything answers in one cycle or it cannot be attached
the UART	no FIFO. The receiver holds exactly one byte and reports an overrun if a second arrives before software takes the first

Lesson 16 — Pipelining

The same work, cut into five pieces



A bubble is not a special instruction: it is `valid` going low with every state-changing control signal forced off.

And a squashed instruction raises nothing — the bug this lesson hit was an illegal-instruction latch that had not been told.

Five stages, four pipeline registers, and everything that goes wrong between them: two forwarding paths, one bypass, one stall, one flush.

Registers between the pieces, so five instructions can be in the machine at once and the clock period only has to cover the longest **piece** instead of the whole chain. Lesson 13′s core did all of it between one clock edge and the next and measured 61.24 MHz; as a computer with real memory it measured 28.

Nothing in this lesson makes any single step faster. The clock goes up because no step has to finish inside the same period as all the others.

A pipeline is not a different processor. It is the same parts in a different arrangement: the ALU is still lesson 10′s, the register file lesson 11′s, the decoder and immediate generator lesson 12′s, the branch comparator and aligners lesson 13′s. Not one of those files is touched — which is why lesson 14′s proof of the ALU still holds, unchanged and un-rerun.

It costs three things, and they are the whole of the core.

Registers. Lesson 13′s entire core had 32 flip-flops: the program counter. This one has 414. That difference *is* pipelining.

Hazards. Instruction *N+1* may need a result instruction *N* has not written yet. Forwarding, stalling and flushing, in that order of preference.

CPI above 1. The single-cycle core retired exactly one instruction per cycle by construction. This one manages it only when nothing goes wrong.

The memory's output register is the IF/ID register

A block RAM's output is registered inside the block — lesson 08′s rule, and the reason lesson 15′s asynchronous memory cost 2,756 LUT4 and not one `DP16KD`. A pipelined core asks in one stage and uses the answer in the next, which is exactly the shape a block RAM offers.

So the pipeline does not merely tolerate the memory's latency. **It is the reason the latency is free.** The register holding the fetched instruction for the whole of ID is the block RAM's own output register, which costs nothing because the block RAM has it either way — and its clock enable is how the core stalls.

Hazards

Forwarding

Two places already hold a value the ALU is about to need: the `EX/MEM` register, holding the instruction one ahead, whose ALU result is computed; and `MEM/WB`, holding the instruction two ahead — including a *loaded*

word, which exists nowhere else yet. `EX/MEM` wins when both match, because the nearer instruction is the more recent one.

Watch out. The MEM-stage value of a **load** is not its result, it is its *address*. Forwarding that gives the program a pointer where it wanted a value, and it computes confident nonsense. The stall below is what prevents it, and the core carries a simulation-only assertion that fires if it ever does not.

The bypass lesson 11 predicted

Lesson 11's register file has no write-to-read bypass, and its comment said exactly why that was right then and wrong now: bypass becomes necessary when a read and a write belong to *different* instructions in flight at once.

Here is that case. An instruction in WB writes its result at the end of a cycle; an instruction in ID reads the register file during that cycle and gets the old value. They are four instructions apart — too far for EX forwarding, too close for the write to have landed. One multiplexer on each read port fixes it.

The one forwarding cannot fix

A load's result does not **exist** until WB: the memory is synchronous, so the word asked for in MEM arrives in the cycle after. An instruction needing it one instruction later would have to read it a cycle before it exists, and no amount of wiring moves a value backwards in time.

So: stall one cycle. That is the load-use hazard, it costs exactly one bubble, and it is why compilers bother to schedule an unrelated instruction into the slot after a load. A bubble, incidentally, is not a special instruction — it is `valid` going low with every state-changing control signal forced off.

Control

Branches resolve in EX, using the *forwarded* operands, which is why resolving them in ID would need a second forwarding network. Resolving in EX costs **two** wrong instructions on every taken branch. Resolving in ID would cost one. Predicting would cost none, when right.

The bug this lesson actually hit

The C program from lesson 15 halted on an illegal instruction a few instructions after printing its echo. The instruction was `0x20737362`, which is ASCII `"bss "`.

The machine had decoded a string constant.

The last instruction of `main()` is a `jal`, and the word after it is the first four bytes of `.rodata`. The pipeline fetches it — of course it does; it does not know the jump is taken until EX — and throws it away correctly, because the flush keeps it from ever reaching EX. But an exception is not something an instruction *does*. It is something it **is**, and the illegal-instruction latch was looking at it anyway.

A bug this course hit. The fix is `&& !redirect`, and the rule behind it is one every real processor obeys: **an instruction that is squashed raises nothing**. Real pipelines carry an exception down the stages with its instruction and raise it at commit, for exactly this reason — by then they know whether it was ever going to run.

Lesson 13's core could not have had this bug: it never fetches speculatively, so every instruction it decodes is one it executes. This is not a bug in trap logic. It is a *control hazard* wearing a disguise — and it is why the conformance program passed while the C did not. Hand-written assembly has a real instruction after every jump; a compiler puts its string table there.

What it cost

MEASURED `cpu5` — the pipelined core

```
make synth M=cpu5
```

resource	count
LUT4	2,110
TRELLIS_FF	414
CCU2C	95
PFUMX	597
L6MUX21	279
TRELLIS_DPR16X4	32

Against lesson 13's core at 1,765 LUT4 and **32** flip-flops. The pipeline cost 345 LUT4 and 382 flip-flops — a 13-fold increase in state for a 20% increase in logic, which is the shape of the bargain.

MEASURED `memory_sync` — the same 2 KB as lesson 15

```
make synth M=memory_sync
```

resource	count
LUT4	121
TRELLIS_FF	117
DP16KD	2

Lesson 15's version of this memory cost **2,756 LUT4** and 512 distributed RAM cells. The difference between the two files is that one reads combinationally and this one does not.

MEASURED the whole computer, against lesson 15's

	lesson 15	lesson 16
LUT4	4,821	**2,538**
TRELLIS_FF	146	**680**
DP16KD	0	**2**
TRELLIS_DPR16X4	544	32

The pipelined computer is smaller. A more complex core, four sets of pipeline registers, a forwarding network and hazard logic — and 2,283 *fewer* LUT4, because the memory moved into two block RAMs out of 208. The pipeline cost 345 LUT4; the memory it made possible paid that back seven times over.

And the memory now scales, which it could not before: at 16 KB the design uses 2,521 LUT4 and 16 of the 208 block RAMs — seventeen more LUT4 for eight times the memory. In lesson 15, merely doubling 2 KB to 4 KB cost 2,442 LUT4 and broke timing.

What it cost in time, and what it bought

Both machines placed and routed three times with different placer seeds, at the same 25 MHz constraint, so the numbers compare.

MEASURED the clock nearly doubles

lesson 15 — single-cycle computer	27.28 / 28.21 / 28.00 MHz
lesson 16 — pipelined computer	**49.48 / 47.52 / 45.74 MHz**

Now the other half of the arithmetic. The conformance program runs 109 instructions in **148** cycles here, against 109 on the single-cycle core: CPI 1.358 against 1.000.

The C delay loop is the clearer case, because it is small enough to count by hand — six instructions, a hundred iterations, and the program's own stopwatch reports 606 cycles on the single-cycle machine and 1,207 here.

MEASURED every cycle of one loop iteration

six instructions	6
`lw` → `addi` load-use stall	1
`lw` → `bge` load-use stall	1
taken `bge`, two instructions flushed	2
taken `jal`, two instructions flushed	2
per iteration	**12**

12 × 100 = 1,200, plus 7 cycles of loop entry and pipeline fill: 1,207.

So is it faster? Not always

Time is cycles divided by clock, and both changed. Taking the slowest seed for each machine:

MEASURED two programs, opposite answers

program	lesson 15	lesson 16	
conformance, 109 instructions	3.89 µs	3.24 µs	**1.20× faster**
the `volatile` delay loop	21.6 µs	26.4 µs	**0.82× — slower**

Pipelining made one program 20% faster and the other 18% slower. The clock went up 1.63×; the delay loop's CPI went up 1.99×, and 1.99 is more than 1.63.

That loop is the worst case on purpose — every iteration is a load immediately followed by its use, twice, and two taken branches — and it is also exactly the shape of code a naive benchmark contains. The honest summary is that pipelining buys *permission to raise the clock*, and whether that is a win depends on the program. Which is not what the textbook diagram suggests.

On the board, where the clock is yours to set

A pipeline running at the same clock is a **slower machine**: the same instructions in more cycles, each cycle the same length. Everything pipelining gives you arrives as permission to raise the clock, and if you do not raise it, you have paid for the hazard logic and bought nothing.

Lesson 15's machine runs at the board's 25 MHz oscillator because it cannot go faster. This one closes at 46, so the board top instantiates lesson 04's PLL — the same file, unchanged — and runs the computer at **40 MHz**. Forty and not forty-six: the slowest placer seed measured 45.74, and taking 46 because one run said 46 is how you ship a machine that works on the bench and fails in July. The shipped build closes at 48.55 MHz against its 40 MHz constraint.

The UART divisor comes from a parameter, so it follows the PLL with nothing else changed, which is why lesson 09 made it a parameter.

Load lesson 15's bitstream, reset, read the cycle count the program prints. Load this one, reset, read it again: **it is larger**, on a machine that is faster. That is the whole lesson in two numbers on a screen.

Lesson 17 — CSRs, traps and the M extension

What a core that halts cannot do

Lesson 13′s core, and lesson 16′s, do the same thing with an instruction they do not understand: **they stop**. The program counter freezes, a light comes on, and that is the end of the machine.

Real processors trap: save where they were, record why, and jump to an address the software chose. The difference between a core that halts and a core that could run an operating system is almost entirely this lesson, and it is smaller than you would think.

A trap is the only mechanism by which software gets control of a machine it is not currently running on. System calls, page faults, debuggers, timer-driven scheduling, and every "something went wrong" message you have ever seen are the same four registers and one jump.

This lesson is genuinely optional — the course's goal is met at lesson 16. It is here because *a correct RV32I core* and *a core that could run an RTOS* are separated by exactly these pieces.

Four additions to lesson 16′s core

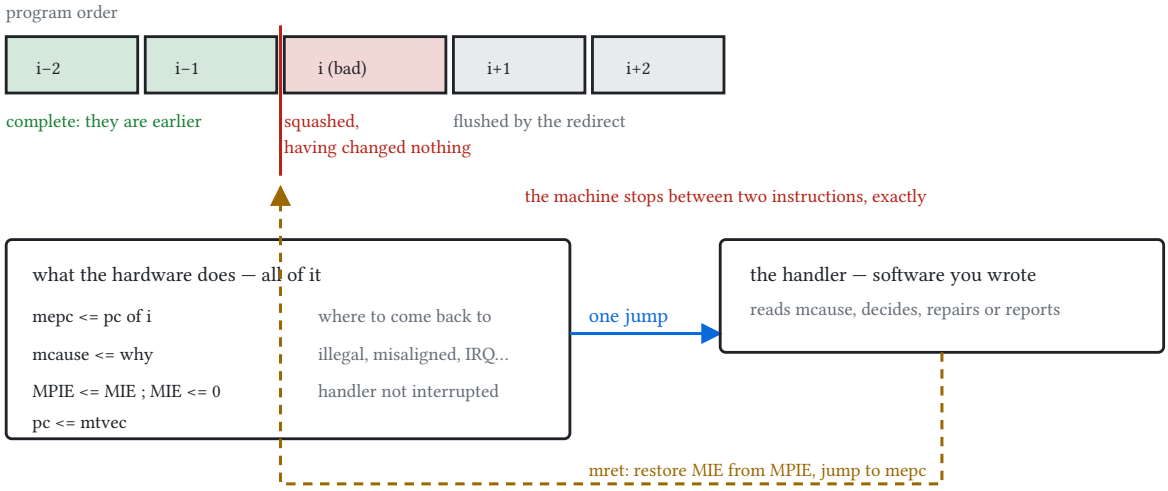
A second decoder

Lesson 12′s decoder recognises nine opcodes and calls everything else illegal, which includes every instruction this lesson needs. A second decoder claims them, and the core computes `illegal = base_illegal AND NOT ext_legal`. That is the smallest honest change: lesson 12′s decoder is a finished, measured artifact, and a lesson that quietly rewrites an earlier one invalidates its numbers.

`MUL` is opcode `0110011` — the same as `ADD` — distinguished only by `funct7 = 0000001`. RISC-V does that deliberately: opcodes are scarce, and an extension needing its own opcode is one that cannot coexist with another. Lesson 12′s `funct7` check is exactly what makes `MUL` illegal on a core without M, which is what the specification requires, and why that check was worth writing. **An extension is a decoder.**

Traps, and why they are precise

Nine registers hold the privileged state. Taking a trap is a redirect with bookkeeping: save the PC in `mepc`, say why in `mcause`, copy `MIE` into `MPIE` and clear `MIE` so the handler is not immediately interrupted, and jump to `mtvec`. **Four registers and a jump** — that is all the hardware does.



Four registers and a jump. System calls, page faults, debuggers, timer scheduling and every "something went wrong" message you have seen are this mechanism — the only way software gets control of a machine it is not running on.

A precise trap. Earlier instructions finish, the trapping one is squashed, four registers are written, and `mret` comes back to the boundary.

Traps are decided in ID, before the instruction has done anything. So everything ahead of it in the pipeline completes, because those instructions are earlier in program order; the trapping instruction is squashed,

having changed nothing; and everything behind it is flushed by the redirect. The machine stops between two instructions, exactly, with `mepc` naming the boundary.

That property is **precision**, and it is not optional: a handler that cannot tell what has happened cannot return from it. It is also the hardest thing about traps in a deep pipeline, and the reason real cores carry an exception down the stages with its instruction and raise it at commit.

Watch out. Taking a trap requires `!redirect` — lesson 16's rule about squashed instructions. There it stopped the machine halting on a string constant; here it is what keeps traps precise, because an instruction on the wrong path of a branch must not trap. It is not going to run.

Interrupts

Everything above is caused by an instruction. The machine timer is not: its pending bit goes high because *time passed*, and that is the entire difference between an exception and an interrupt. It arrives as one input wire.

Watch out. The timer's interrupt is a **level**, not an event: it stays asserted while `mtime ⩾ mtimecmp`. A handler that returns without moving `mtimecmp` forward is re-entered the instant `MRET` re-enables interrupts, forever. This is the classic first bug with a timer.

Adding the timer to the system cost one line — a select line and an entry in the read multiplexer. The address decoder, the compiler and the linker script are all unchanged. That is what a memory map buys.

An EX stage that can take longer than a cycle

Multiply and divide do not finish in a cycle, so for the first time in this course EX can say "not yet" — which needs a stall that holds the instruction *in* EX rather than ahead of it.

stall	who waits	what EX gets
load-use (lesson 16)	the instruction in ID	a bubble
multiply or divide	the instruction in EX	nothing — it holds

Both are one signal to the rest of the pipeline, which is why adding the divider beside the multiplier changed nothing anywhere else.

The bug this lesson actually hit

The program printed its greeting twice.

```
csrw  mtvec, a5     # in EX, writing mtvec at the end of this cycle
ecall               # in ID, reading mtvec to find out where to go
```

Both in the machine at once, one stage apart. The `ECALL` read the *old* `mtvec` — zero, because nothing had set it yet — and jumped to address zero, which is the first instruction of the program. The machine restarted itself, printed its greeting again, set `mtvec` again, and the second `ECALL` worked. On the board that would look like a glitch.

A bug this course hit. Lesson 16's forwarding network cannot help, and the reason is worth more than the fix. Forwarding moves **values** between instructions. This is not a value: it is a change to how the machine *behaves* — where traps go, whether interrupts are allowed, what the timer is compared against. There is no wire to forward "the rules are different now" along.

So the pipeline serialises: while a CSR instruction is in EX, nothing is decoded behind it. One bubble per CSR instruction — the right price for an instruction a program executes a few dozen times, and the reason real instruction sets describe CSR writes as *serialising* rather than as ordinary register writes.

A handler in C

```
__attribute__((interrupt("machine")))
void trap_entry(void)
```

That attribute asks gcc for a function that can be jumped to by **hardware** rather than called by software. It saves and restores every register it touches — a normal function may clobber the temporaries freely, this one may not clobber anything — and it returns with `MRET` instead of `RET`, which restores the interrupt enable and

jumps to `mepc`. Disassemble it and count the stores in its prologue: that stack traffic is the cost of an interrupt, before any of your code runs.

`mepc` points *at* the faulting instruction, not after it — right for a page fault, where you fix the problem and re-run it, wrong for `ECALL`, where re-running would trap again immediately. The handler decides, which is the point.

There is no C syntax for a control and status register, and no reason there should be: they are not memory, they have no addresses C can form, and the instructions take the CSR number as an *immediate*, so it must be a compile-time constant. Hence macros that paste the name into inline assembly. This is the floor of the C abstraction, and every operating system has a header that looks like it.

One flag, two extensions

`-march=rv32im_zicsr`. The `m` makes `a * b` one instruction instead of a call into libgcc. The `zicsr` makes `csrr` assemble *at all* — the CSR instructions were part of the base integer ISA until 2019, when they were split into their own extension, because a core needing no privileged state should not have to implement six instructions to be called RV32I. The hardware does not care: the decoder would have claimed them under any name.

What it cost

MEASURED `soc17_top` — the whole machine
`make synth`

resource	count
LUT4	3,866
TRELLIS_FF	1,285
CCU2C	366
PFUMX	896
L6MUX21	340
TRELLIS_DPR16X4	32
DP16KD	16
MULT18X18D	4
EHXPLLL	1

3,866 of the 85F's 83,640 LUT4 — 4.62% of the part.

Four hard multipliers and sixteen block RAMs, for a computer with 16 KB of memory that takes interrupts.

MEASURED where the additions went, each synthesised on its own

module	LUT4	TRELLIS_FF	MULT18X18D	
cpu5t	3,270	887	4	this lesson's core
cpu5	2,110	414	0	lesson 16's, for comparison
csr	1,402	225	0	nine registers and the trap state
div_unit	528	169	0	32 cycles of restoring division
mul_unit	36	68	**4**	the DSPs do the work
timer	164	128	0	two 64-bit registers, one comparator
decode_ext	56	0	0	every instruction this lesson added

The whole privileged architecture is a decoder of 56 LUT4 and a register file of about 1,400. Traps are cheap; what they enable is not, but the mechanism itself is smaller than the ALU.

Multiply costs 36 LUT4 and four DSP blocks; divide costs 528 LUT4 and no DSP at all. That asymmetry is the whole of the M extension in two rows: multiplication is a block the chip already has, and division is a

state machine you have to build. It is why `a / b` is expensive in C and `a * b` is not, and why compilers turn division by a constant into a multiply by a magic number.

`cpu5t` is 1,160 LUT4 and 473 flip-flops more than `cpu5`. The modules above sum to more than that, because a module synthesised alone must build every output it exposes — the same effect lesson 13 measured when `cpu` came out larger than the `cpu_top` containing it.

What it cost in time

This core **does not close timing at lesson 16's 40 MHz**, and finding out why took three builds and nine place-and-route runs.

MEASURED three versions of one multiplier

build	constraint	three placer seeds	verdict
combinational multiplier	40 MHz	34.5 / 34.9 / 35.5	fails all three
product registered	40 MHz	41.1 / 40.1 / **39.6**	fails one in three
…and operands registered too	40 MHz	38.5 / 39.8 / 41.5	fails two in three
shipped: product registered	**35.71 MHz**	**39.7 / 39.8 / 39.5**	**passes**

Built combinationally, the 32×32 multiplier *was* the critical path of the machine, and nextpnr named it exactly: 29 ns from the block RAM's output, through the forwarding network, the four DSPs and the carry chain that sums their partial products, into the `EX/MEM` register. Lesson 16's core without it closed at 46 MHz.

Registering the product fixed most of that, and the multiply became a two-cycle instruction — which is what real cores do, and why every instruction-timing table lists multiply latency separately from throughput.

The third row is the one worth having. Registering the *operands* as well is the textbook next step: split the long path in two, get a faster clock. It bought **nothing** — the same clock within the placer's own spread — and cost an extra cycle on every multiply. The multiplier had stopped being the critical path after the first register, and a second one optimises something that is no longer the problem.

The only reason anyone knows that is that it was measured three times rather than once. A single run of this design lands anywhere between 38.5 and 41.5 MHz depending on a random seed, and either number alone tells a confident, wrong story.

So the board runs at 35.7 MHz, from this lesson's own PLL, about 10% below the slowest measured placement — the same kind of margin lesson 16 leaves at 40.

MEASURED the three computers in this course

	clock	why
lesson 15 — single-cycle	25 MHz	it closes at 28; the oscillator is 25
lesson 16 — pipelined	40 MHz	closes at 45.7, worst seed
lesson 17 — with traps and M	**35.7 MHz**	closes at 39.5, worst seed

Privileged state and hardware multiply cost about six megahertz. That is the price of this lesson, stated the way this course states prices.

What this machine still does not do

modes	machine mode only. No user mode, no supervisor mode, no virtual memory — so no protection: every instruction can reach every CSR
CSRs	an unimplemented CSR reads as zero. The specification says it should raise an illegal-instruction exception, and a core meaning to run an operating system must do that
misaligned access	still not detected, as in lesson 13. The aligner does something well-defined and wrong; a real core traps
interrupts	the timer, and nothing else. No external interrupt controller, no software interrupt, no nesting — a handler runs with interrupts disabled until it returns
compliance	the official `riscv-arch-test` suite has not been run against this core. What is claimed is what was measured: lesson 13′s conformance program gives identical results, and the lesson 14 proof of the ALU still holds

So: a machine that traps, handles interrupts, multiplies in hardware and counts its own cycles — built from a single logic gate in seventeen steps.

APPENDICES

Reference

Every measurement in the book in one place, the style rules every lesson follows, and where to go next.

A — Numbers that matter	Every count and every frequency in the book, in one set of tables
B — Verilog style rules	The conventions the lessons follow, and the mistake each one prevents
C — Where to go next	Specifications, books, cores worth reading, and what was left out

Appendix A — Numbers that matter

Every measurement in this book, in one place. The lessons give the context and the reasoning; this is for the reader who wants the shape of the costs without the narrative, or who wants to check their own build against the course's.

All counts are `make synth` — `synth_ecp5` plus `stat`, not a placed design. All frequencies are `make bit`, where `nextpnr` reports the routed design's maximum for each clock. The exact toolchain versions are in *Tools and Workflow* a different release can move a LUT4 count by a few percent.

The part

resource on the 85F	count
LUT4	83,640
Flip-flops	83,640
Block RAM `DP16KD`	208 × 18 Kbit
Multipliers `MULT18X18D`	156
PLLs `EHXPLLL`	4
Board clock	25 MHz

Whole designs, lesson by lesson

Each lesson's top module, as built — the figure that answers "how much of the part did that take".

lesson	top module	LUT4	TRELLIS_FF
00	`blink` — a 26-bit counter	1	26
01	`gates_top` — eleven outputs, no state	7	—
02	`comb_top`	14	—
03	`seq_top`	24	29
04	`clocks_top` — with a PLL	19	67
05	`fsm_top`	75	113
06	`oled_top` — the display controller	108	72
07	`text_top` — font and coordinate math	185	72
08	`mem_top` — two block RAMs	80	61
09	`uart_top` — both halves	91	71
12	`decode_top`	53	14
13	`cpu_top` — core, memories, board wiring	1,268	40
15	`soc_top` — the whole computer	4,821	146
16	`cpu5` — the pipelined core	2,110	414
17	`soc17_top` — with traps and M	3,866	1,285

Lessons 10 and 11 are modules rather than board designs, and lesson 14 proves lessons 10 and 11 rather than building anything.

The processor, part by part

lesson	module	LUT4
10	`alu` — the whole thing	885
10	`adder` — 32-bit addition (`CCU2C` only)	0 (17 `CCU2C`)
10	`shift_barrel` — single-cycle shifting	667
10	`shift_iterative` — the same shifts, 31 cycles	76
11	`regfile` — combinational read, as RV32I needs	266
11	`regfile_bram` — the same file, asking for BRAM	214
12	`immgen` — the immediate generator	70
12	`decode` — the control decoder	45
13	`cpu` — the processor itself	1,765
15	`memory` — 2 KB, two asynchronous read ports	2,756
15	`busdec` — the entire memory map	20
15	`mmio` — UART, LEDs, buttons, stopwatch	200
16	`memory_sync` — the same 2 KB, synchronous	121 (2 `DP16KD`)

Two of those lines are the course's sharpest cost lessons. The barrel shifter is 80% of the ALU — 667 LUT4 against the adder's 17 carry cells — because the fabric has dedicated carry hardware and no dedicated shift hardware. And 2 KB of memory costs 2,756 LUT4 when read asynchronously and 121 LUT4 plus two block RAMs when read synchronously: the same capacity, a factor of twenty apart, decided by one clock edge.

Where the hand-built version lost

the same function, two ways	the expensive way	the cheap way
32-bit adder (lesson 02)	138 `LUT4` + 74 mux	17 `CCU2C`
16-way decode as `case` vs. array (lesson 02)	7 `LUT4`	7 `LUT4`
4-state FSM, one-hot vs. the tool's choice (lesson 05)	41 `LUT4`	11 `LUT4`
Moore vs. Mealy sequence detector (lesson 05)	5 `LUT4` (Mealy)	2 `LUT4` (Moore)
1 KB font ROM: logic vs. block RAM (lesson 08)	9 `LUT4` (32 bytes)	1 `DP16KD` (1 KB)

Maximum frequency

design	constraint	achieved
32-bit ALU alone (lesson 10)	25 MHz	115.94 MHz
Single-cycle core, small memories (lesson 13)	25 MHz	61.24 MHz
The same core as a computer (lesson 15)	25 MHz	27.28 / 28.21 / 28.00 MHz
— with 4 KB of memory instead of 2 KB	25 MHz	25.14 / 24.44 / 24.40 MHz, **fails 2 runs in 3**
— with 1 KB	25 MHz	31.73 / 30.63 / 30.49 MHz
Pipelined computer (lesson 16)	25 MHz	49.48 / 47.52 / 45.74 MHz
— as shipped, at its own PLL clock	40 MHz	48.55 MHz
With traps and M (lesson 17)	35.7 MHz	39.7 / 39.8 / 39.5 MHz
— the same core asked for 40 MHz	40 MHz	34.5 / 34.9 / 35.5 MHz, **fails all three**

Three seeds, where three are given, because place-and-route is randomised and a single run is not a measurement. Lesson 17's shipped 35.7 MHz is about 10% below the best seed's result, which is the margin that makes the build reproducible rather than lucky.

What pipelining bought and cost

	lesson 15, single-cycle	lesson 16, pipelined
clock	28 MHz	45.7 MHz worst seed (1.63×)
CPI, conformance program	1.000	1.358
CPI, the C delay loop	1.000	1.99×
`LUT4` (core only)	1,765	2,110
`TRELLIS_FF` (core only)	32	414
net effect	—	one program 20% faster, the other 18% slower

The delay loop is the worst case on purpose — every iteration is a load immediately followed by its use, twice, and two taken branches. Pipelining buys *permission to raise the clock* whether that is a win depends on the code.

Flip-flops, accounted for

Lesson 05 reports 113 flip-flops, and the whole of the course's method is that they decompose exactly:

one-second tick divider (25 bits)	25
traffic light: 2 state + 3 timer	5
four debouncers	76
four edge detectors	4
lock state	3
total	**113**

The state machines are 8 flip-flops out of 113. The rest is debouncing and timekeeping — the plumbing needed to talk to the physical world. That ratio is normal, and worth seeing once.

Appendix B — Verilog style rules

Conventions every lesson follows. Each exists because it corresponds to a mistake that is easy to make and annoying to debug.

Synchronous design, one clock

Everything is clocked by `clk_25mhz`. No gated clocks, no latches, no asynchronous resets unless a lesson is explicitly about them.

If you need something slower, divide with a counter and use the result as an **enable**, not as a clock:

```
// Yes: one clock domain, a slow enable.
reg [15:0] div = 0;
wire tick = (div == 16'd49999);
always @(posedge clk) div   <= tick ? 16'd0 : div + 1'b1;
always @(posedge clk) if (tick) count <= count + 1'b1;

// No: a derived clock. Does not use the clock network, will not meet
// timing predictably, and makes the design unanalysable.
always @(posedge div[15]) count <= count + 1'b1;
```

Blocking versus non-blocking

- `<=` inside `always @(posedge clk)` — always, without exception.
- `=` inside `always @(*)` combinational blocks.

Non-blocking assignments all take effect at the end of the time step, which is what a bank of real flip-flops does. Blocking assignments take effect immediately, which chains them.

```
// Shift register: each stage sees the OLD value of the previous one.
always @(posedge clk) begin
    a <= in;  b <= a;  c <= b;      // three flip-flops
end

// With blocking assignments this collapses: a, b and c all become `in`.
always @(posedge clk) begin
    a = in;   b = a;   c = b;       // ONE flip-flop, replicated
end
```

Lesson 03 measures it: three flip-flops against one.

Do not infer latches

A combinational `always` block must assign every output on every path, or synthesis infers a latch — storage with no clock, which is almost never what you wanted.

```
// Latch: `y` keeps its old value when sel is 2'b11.
always @(*) case (sel)
    2'b00: y = a;  2'b01: y = b;  2'b10: y = c;
endcase

// Fixed: assign a default FIRST, then override.
always @(*) begin
    y = 1'b0;
    case (sel)
        2'b00: y = a;  2'b01: y = b;  2'b10: y = c;
        default: y = 1'b0;
    endcase
end
```

Watch out. In a *clocked* block the same omission is correct. `if (en) q <= d;` with no `else` is a flip-flop with an enable, not a latch, because a flip-flop already holds its value between edges. Same syntax, opposite meaning, decided by whether the block is sensitive to a clock edge or to its inputs.

Widths

State widths explicitly and keep them consistent. Verilog will silently zero-extend, truncate or sign-extend, and the result is a bug that appears only at a boundary value.

```
reg  [7:0] count;
count <= count + 1'b1;        // explicit 1-bit literal, not bare `1`
wire [7:0] sum   = a + b;     // drops the carry — deliberate? say so
wire [8:0] sum_c = a + b;     // keeps it
```

Two traps worth naming, both silent:

Shifts	`a << b` is as wide as `a`. So `1'b1 << sel` shifts a one-bit value and is always zero. Widen the literal before shifting.
Carries	`a + b` into a W-bit destination throws the carry away. Widen the intermediate to W+1.

And when a truncation is intended, say so — widen, then slice:

```
localparam [31:0]   LAST32 = DEPTH - 1;
localparam [AW-1:0] LAST   = LAST32[AW-1:0];
```

Reset and initialisation

The ECP5 initialises flip-flops from the bitstream, so `reg x = 0;` works and is used throughout this course. That is a convenience of this family, not a portable habit — on an ASIC you need an explicit reset. Where a lesson needs a real reset, it is synchronous:

```
always @(posedge clk)
    if (rst) count <= 0;
    else     count <= count + 1'b1;
```

One driver per signal

A register is driven from exactly one `always` block, and priority between its inputs is expressed with `if`/`else`. Two blocks assigning one register have no defined ordering in simulation and force synthesis to guess — Verilator rejects it, correctly.

Naming

`clk_25mhz`	the board clock, named for its frequency
`*_n`, `*n`	active low — `oled_resn`, `csn`
`*_i`	an internal copy of an output you also need to read
`S_*`	state constants, as `localparam`
`tb_*`	the testbench for the module of the same name
`*_next`	the combinational next-state value of a register

Name states as constants, never bare numbers in a `case`. `2'd2` tells a reader nothing; `S_GREEN` tells them everything.

Testbenches

Every testbench ends with an explicit verdict:

```
if (errors == 0) $display("PASS: %0d vectors checked", n);
else             $display("FAIL: %0d of %0d wrong", errors, n);
```

`make sim-all` greps for `PASS`, so this is the regression suite's interface rather than decoration.

Three further rules, each learned the hard way in a lesson:

1. **Compute the reference a different way from the design.** A model that shares the design's mistake proves nothing. Lesson 10's shifts are checked against an independent iterative implementation for exactly this reason.
2. **Set up monitors before stimulus.** Lesson 09's bit-period measurement asserted `start` and then waited for an edge that had already passed. Use `fork`/`join`.
3. **Do not assert what you have not established.** Lesson 09's testbench claimed mistimed frames were never silently corrupted, and passed. The claim was false. A test that asserts something false and passes is worse than no test.

Mutation testing

A testbench that has never caught a bug is unproven. Break the design deliberately and check the test notices. A surviving mutation means one of two things, and distinguishing them is the skill:

the test is incomplete	lesson 12: every store offset in the vectors happened to be even, so a bug in the immediate's low bit was invisible
the code is redundant	lesson 11: `x0` is enforced on both the read and the write path, so removing either alone changes no behaviour

Appendix C — Where to go next

Specifications worth having open

The RISC-V ISA manual, volume 1	Unprivileged instructions. Free, readable, and short enough to actually read — which is unusual for an ISA specification and is part of why this course can end where it does.
The RISC-V ISA manual, volume 2	Privileged architecture: the CSRs, traps and the machine timer that lesson 17 is about.
Lattice ECP5 family datasheet	The authoritative numbers for the part: setup and hold times, block RAM configurations, PLL ranges.
SSD1331 datasheet	The display controller of lessons 06 and 07. Worth reading once as an example of the genre.

Books

Patterson & Hennessy, *Computer Organization and Design*, RISC-V edition	The standard text, and the one this course's lessons 10 through 16 run parallel to. Where this book measures, that one explains — they complement each other well.
Harris & Harris, *Digital Design and Computer Architecture*	Starts further back and covers the Verilog alongside the architecture. Closest in spirit to this course.
Nisan & Schocken, *The Elements of Computing Systems*	The Nand2Tetris book. Builds a complete machine from NAND upward in simulation, and then a compiler and an operating system on top of it. A different and excellent answer to the same question.

Projects to read

picorv32	A small, well-documented RV32I core by Claire Wolf. The natural next thing to read after finishing lesson 13 — a production-quality version of what you just built.
SERV	The world's smallest RISC-V core, by Olof Kindgren — bit-serial, around 200 LUTs. A superb demonstration that "correct" and "fast" are independent axes.
VexRiscv	A configurable RV32 core generator. Shows what a pipeline, caches and an MMU look like at production scale.
LiteX	An SoC builder. Where you go when you want a CPU *and* SDRAM, Ethernet, video and an SD card, without building all of them yourself.
The ULX3S examples	The ULX3S project's own repository — SDRAM controllers, video output, emulators of older machines. Written for this exact board.

Things this course deliberately left out

Each would make a good next project:

1. **An SDRAM controller.** The board has 32 MB of it, 16 bits wide, and it needs refresh, row activation and careful timing. The single hardest thing on this board, and the thing that turns a toy into a computer.
2. **Digital video output.** Serialisation at ten times the pixel clock, which needs the PLL of lesson 04 used in anger.
3. **Caches.** Lesson 16's pipeline makes memory the bottleneck; a cache is the answer, and cache coherence is where it gets genuinely hard.

4. **A compiler back end.** Lesson 15 uses GCC. Targeting your own instruction set with LLVM is a large project and an education in why ISAs look the way they do.
5. **Formal proof of the pipeline.** Lesson 14 proves the single-cycle core. Doing the same for lesson 16′s pipelined version is strictly harder and strictly more valuable, because that is where the subtle bugs live.

A closing thought

The most transferable thing in this course is not Verilog, and it is not RISC-V. It is the habit of asking *what did that actually cost* and then running the two-second command that answers it.

Four times in these lessons, a rule that everyone knows turned out to be wrong for the fabric in front of us — and in each case the measurement took less time than arguing about it would have. That habit works on tools you have not met, on chips that do not exist yet, and on problems that are not about hardware at all.